



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Titolo della Tesi

RELATORE

Prof. Fabio Palomba

Dott. Nome tutor

Università degli Studi di Salerno

CANDIDATO

Marco Donatiello

Matricola: 0512119045

Anno Accademico 2025-2026

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

Abstract

Il testo dell'abstract qui...

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	v
1 Introduzione	1
2 Analisi dello stato del arte e contesto tecnologico	2
2.1 OWASP Cloud Security Top 10	2
2.1.1 IaC misconfiguration	4
2.2 API Security Risks	6
2.2.1 API1:2023 – Broken Object Level Authorization	8
2.2.2 API2:2023 – Broken Authentication	15
2.2.3 API3:2023 – Broken Object Property Level Authorization	16
2.2.4 API4:2023 – Unrestricted Resource Consumption	19
2.2.5 API5:2023 – Broken Function Level Authorization	20
2.2.6 API6:2023 – Unrestricted Access to Sensitive Business Flows	23
2.2.7 API7:2023 – Server Side Request Forgery	24
2.2.8 API8:2023 – Security Misconfiguration	26
2.2.9 API9:2023 – Improper Inventory Management	28
2.2.10 API10:2023 – Unsafe Consumption of APIs	29
2.3 Sintesi: dove si trova il difetto	31

3	Metodo di ricerca	33
3.1	Approccio metodologico	33
3.1.1	Obiettivi e domande di ricerca	33
3.1.2	Impostazione dello studio	34
3.1.3	Il criterio di classificazione	35
3.1.4	Requisiti derivati per lo strumento di rilevazione	36
3.1.5	Disegno della valutazione	37
3.2	Rilevamento delle IaC misconfiguration	38
3.2.1	Architettura del sistema	39
3.2.2	Lo scanner Checkov e la traduzione nel modello di dominio .	41
3.2.3	Correlazione e punteggio di rischio	43
3.3	Categorie ibride: analisi statica e residuo dinamico	46
3.3.1	API1 – Il legame fra oggetto e chiamante	49
3.3.2	API2 – L'identità del chiamante e la validità dei token	54
3.3.3	API3 – Le proprietà protette di un oggetto	57
3.3.4	API5 – Il ruolo richiesto per invocare una funzione	60
3.4	Categorie tecniche: un metodo per categoria	64
3.4.1	API4 – I limiti sul consumo di risorse, su tre livelli	64
3.4.2	API7 – Dall'input dell'utente alla chiamata di rete	66
3.4.3	API8 – La configurazione confrontata con i valori di riferimento	68
3.4.4	API10 – La fiducia nelle risposte dei servizi esterni	70
4	Risultati	73
4.1	Oggetto della valutazione	73
4.2	Risultati sui casi di prova	75
4.3	Risultati su codice non scritto per i moduli	77
4.4	Comportamento delle categorie	79
4.4.1	Categorie ibride	79
4.4.2	Categorie a rilevazione statica	80
4.4.3	Categorie senza modulo	81
4.5	Limiti emersi dalla valutazione	82
4.6	Minacce alla validità	85

5 Conclusioni	87
----------------------	-----------

Bibliografia	88
---------------------	-----------

Elenco delle figure

3.1	Diagramma dei componenti del sistema	39
3.2	Diagramma delle classi del dominio e del motore di correlazione . .	40
3.3	Diagramma di attività dell'adapter Checkov	44
3.4	Diagramma di sequenza della pipeline statica	45

Elenco delle tabelle

3.1	Policy Checkov pertinenti	42
3.2	Risoluzione dei controlli nel catalogo delle policy	43
3.3	Batteria di prove dinamiche della Broken Authentication	56
3.4	Regole statiche della BOPLA	59
3.5	Regole del modulo Broken Function Level Authorization	62
3.6	Regole del modulo Server-Side Request Forgery	68
3.7	Regole del modulo Security Misconfiguration	70
3.8	Regole del modulo Unsafe Consumption of APIs	72
4.1	Prove di cui si riportano gli esiti	74
4.2	Punteggi di rischio sul bersaglio Terraform	76

CAPITOLO 1

Introduzione

Alcune linee guida per scrivere la tesi: 1) Ogni immagine/tabella deve avere le seguenti caratteristiche: a) Una caption; b) Deve essere esplicitamente commentata all'interno del testo (ricordatevi che se vi riferite alla figura/tabella inserendo anche il corrispettivo numero e.g., Figura X mostra che... allora la prima lettera deve essere scritta in maiuscolo; E ricordatevi che latex mette a disposizione un comando per citare correttamente (ref) senza dover mettere a mano il numeretto corrispondente) c) Le immagini devono essere rifatte tramite PPT o similari e salvate in pdf prima di inserirle nel file latex, altrimenti rischiate di inserire immagini sgranate. 2) Ricordatevi sempre di citare le fonti e di rifrasare, altrimenti rischiate di incorrere nel reato di plagio (che può comportare anche la revoca del titolo accademico) e che citare non sminuisce il vostro lavoro, anzi, lo rafforza. 3) L'Abstract deve necessariamente contenere le seguenti informazioni a) Un breve riassunto della problematica; b) Una breve descrizione di ciò che è stato fatto; c) I risultati ottenuti.

Analisi dello stato del arte e contesto tecnologico

2.1 OWASP Cloud Security Top 10

Il portale AppSecMaster ha elaborato una propria classificazione dei dieci rischi più comuni e di maggiore impatto nei moderni ambienti cloud, ispirandosi ai principi promossi da OWASP in materia di sicurezza applicativa, che vanno dall'infrastruttura mal configurata alla scarsa visibilità delle risorse in esecuzione. Di seguito è riportata la lista delle principali problematiche individuate:

1. **Security Misconfiguration** — Bucket di storage lasciati con permessi troppo permissivi o istanze di calcolo distribuite senza le dovute restrizioni restano tra le cause più frequenti di violazione dei dati in ambienti cloud.
2. **Identity / Access Mismanagement** — Ruoli IAM assegnati con privilegi superiori a quelli effettivamente necessari, e mai rivisti dopo la configurazione iniziale, espongono i sistemi a un concreto rischio di escalation dei permessi.
3. **Improper Secrets/Key Management** - Se chiavi API e credenziali vengono salvate in chiaro o incorporate nel codice sorgente, la loro compromissione può aprire un accesso laterale a più servizi collegati.

4. **Insecure Network Segmentation** — Senza una reale separazione tra reti e ambienti diversi, un attaccante che viola un singolo componente può muoversi liberamente verso risorse più sensibili.
5. **Injection / Input Validation Flaws in Cloud Services** — Le classiche vulnerabilità di injection assumono forme nuove nei contesti cloud-native, colpendo funzioni serverless, code di messaggi e gateway API dove i controlli sugli input non sono sempre applicati in modo uniforme.
6. **Insecure Deserialization / API Exposure** — La gestione non sicura di dati serializzati, unita ad API interne prive di adeguati controlli di autenticazione o versionamento, costituisce un punto d'ingresso sfruttabile per eseguire codice non autorizzato.
7. **Vulnerable Third-Party/Container Dependencies** — Librerie open source e immagini container non aggiornate accumulano vulnerabilità note, soprattutto quando le stesse basi vengono riutilizzate tra più progetti senza verifiche periodiche.
8. **Improper Asset Inventory / Visibility** — Risorse cloud dimenticate o mai tracciate correttamente, incluse istanze abbandonate e servizi "ombra", sfuggono facilmente al controllo dei team di sicurezza.
9. **Insufficient Logging / Monitoring** — Log distribuiti su decine di servizi diversi e privi di un formato coerente rendono difficile individuare per tempo un'intrusione in corso.
10. **Unvalidated Redirects/Cloud Function Abuse** — Funzioni serverless eseguite con permessi troppo ampi, combinate a controlli insufficienti sui redirect, possono essere sfruttate per aggirare le restrizioni di accesso e ottenere un punto d'appoggio nascosto nell'infrastruttura.

Tra le problematiche elencate, il presente lavoro si concentra sulla prima, indicata nel seguito come IaC misconfiguration¹, e sulla componente di esposizione delle API della sesta, Insecure Deserialization / API Exposure: sono le due voci più direttamente legate agli obiettivi del sistema sviluppato e quelle che ricorrono con maggiore frequenza negli scenari di autorizzazione tipici delle architetture basate su API. La prima è approfondita nella sottosezione seguente; la seconda nella Sezione 2.2, dove la classificazione OWASP dedicata alle API la articola in dieci categorie distinte.

2.1.1 IaC misconfiguration

Una IaC misconfiguration nasce da un errore nella configurazione di un servizio, di un prodotto o di una risorsa infrastrutturale: le impostazioni di sicurezza predefinite non vengono irrobustite a sufficienza, e questo lascia scoperto un fianco che con poco sforzo potrebbe essere protetto. I numeri parlano chiaro: il documento ufficiale OWASP riporta che il 90% delle applicazioni testate è risultato vulnerabile ad almeno una forma di misconfiguration [1]. Nella maggior parte dei casi non c'è nulla di sofisticato dietro: è una dimenticanza, un'impostazione lasciata di default, un permesso che nessuno ha mai rivisto. Ed è proprio questo che la distingue da una vulnerabilità nel codice applicativo — qui il problema non sta nella logica del software, ma in come le risorse sono state predisposte.

Le forme più comuni in cui una misconfiguration si manifesta sono, in fondo, poche e ricorrenti. Un bucket di storage lasciato accessibile pubblicamente, quando avrebbe dovuto restare privato, è probabilmente la più diffusa: basta una policy impostata male, magari copiata da un altro progetto senza troppa attenzione. Un container che gira con privilegi di root, invece, espone il sistema a un rischio diverso ma altrettanto concreto — se l'attaccante riesce a comprometterlo, si ritrova con permessi ben oltre quelli che gli servirebbero per fare danni. C'è poi il caso dei template Infrastructure-as-Code scritti senza le dovute verifiche: un file Terraform o CloudFormation che definisce una risorsa con permessi troppo ampi si porta dietro

¹IaC (*Infrastructure as Code*): pratica per cui le risorse infrastrutturali — reti, istanze, storage, permessi — sono dichiarate in file versionati che uno strumento di provisioning traduce nello stato reale; il file diventa così un artefatto ispezionabile prima che la risorsa esista.

lo stesso errore ogni volta che viene riutilizzato, replicandolo su ambienti diversi senza che nessuno se ne accorga.

Un caso reale: il data breach Capital One Il caso Capital One del luglio 2019 resta probabilmente l'esempio più citato di questa categoria di rischio. Un bucket Amazon S3 lasciato con permessi troppo ampi permise a un'attaccante di sottrarre oltre 100 milioni di record clienti, tra dati anagrafici, indirizzi e informazioni finanziarie sensibili [2]. Nessuna falla nella piattaforma cloud era coinvolta: la responsabilità ricadeva interamente sulla configurazione impostata dal cliente. Non a caso AWS, commentando l'accaduto, ribadì che *"Amazon S3 is secure by default"* [3], scaricando di fatto la colpa sulla gestione dei permessi lato utente. Sul piano economico il conto fu salato: 80 milioni di dollari di multa dall'Office of the Comptroller of the Currency, più 190 milioni versati per chiudere una class action. Capital One dimostra bene quanto una semplice disattenzione in fase di configurazione possa costare quanto, se non più di, un attacco costruito su un exploit tecnicamente elaborato.

Episodi come questo spiegano perché, negli ultimi anni, le organizzazioni abbiano iniziato ad affidarsi sempre meno alla revisione manuale delle configurazioni e sempre più a strumenti automatizzati capaci di individuare questo tipo di errori prima che vengano sfruttati. Una configurazione insicura, del resto, raramente viene introdotta di proposito: emerge quasi sempre da un template riutilizzato senza controllo, da un default mai cambiato, da una revisione saltata per mancanza di tempo — fattori che un controllo manuale sistematico difficilmente riesce a intercettare su larga scala e con continuità.

Il documento ufficiale OWASP illustra questa categoria di rischio attraverso quattro scenari esemplificativi [1]. Il primo riguarda applicazioni di esempio dimenticate su un server di produzione, spesso accompagnate da account amministrativi con credenziali mai modificate rispetto ai valori predefiniti. Il secondo descrive un server su cui la funzionalità di directory listing resta abilitata, consentendo a un attaccante di risalire fino al codice sorgente tramite reverse engineering. Il terzo riguarda messaggi di errore troppo dettagliati restituiti dall'application server, capaci di esporre informazioni sull'infrastruttura sottostante o versioni di componenti vulnerabili.

Il quarto, infine, riguarda permessi di condivisione predefiniti di un cloud service provider lasciati aperti ad altri utenti su Internet, con conseguente esposizione di dati sensibili archiviati nello storage cloud.

2.2 API Security Risks

Le API oggi fanno gran parte del lavoro che una volta faceva il sito web stesso: non restituiscono una pagina pensata per essere letta da una persona, ma dati grezzi destinati a un'altra app, a un altro servizio, a un sistema che poi decide cosa farne. Lo schema delle richieste è prevedibile, gli endpoint² espongono operazioni precise su risorse precise. Per un attaccante questo significa meno lavoro di interpretazione: non deve capire un'interfaccia grafica, gli basta osservare come sono costruite le chiamate.

OWASP mantiene, accanto al Top 10 per le applicazioni web, una classificazione separata dedicata alle API. Alcune categorie compaiono in entrambe le liste — la misconfiguration, il broken access control — ma buona parte dei rischi specifici delle API riguarda un livello che il Top 10 generale non copre: l'autorizzazione su singoli oggetti, proprietà, funzioni. Un'applicazione web tradizionale filtra cosa mostrare all'utente; un'API deve filtrare cosa restituire nella risposta. La differenza sembra minima, ma è da lì che nascono molte delle vulnerabilità trattate in questa sezione.

Le dieci categorie che compongono l'edizione 2023 riflettono questa logica: non rischi generici, ma rischi legati al modo specifico in cui un'API espone dati e funzionalità. Di seguito la lista completa, in ordine di gravità secondo OWASP [4]:

1. **API1:2023 – Broken Object Level Authorization (BOLA)** — un endpoint accetta l'ID di una risorsa senza verificare che chi lo richiede abbia davvero il diritto di accedervi; basta cambiare un numero nell'URL per leggere dati altrui.

²*Endpoint*: il singolo punto di accesso di un'API, identificato da un URL e da un metodo HTTP, che espone una specifica operazione su una risorsa.

2. **API2:2023 – Broken Authentication** — meccanismi di autenticazione implementati male, token che non scadono mai, o password recuperabili senza controlli sufficienti.
3. **API3:2023 – Broken Object Property Level Authorization** — qui il problema non è accedere alla risorsa sbagliata, ma vedere o modificare campi che non dovrebbero essere esposti o scrivibili da quel particolare utente.
4. **API4:2023 – Unrestricted Resource Consumption** — assenza di limiti su chiamate, dimensione dei payload o uso di risorse computazionali, con conseguenze che vanno dal semplice rallentamento a un vero e proprio denial of service.
5. **API5:2023 – Broken Function Level Authorization** — un utente normale riesce a invocare funzioni riservate agli amministratori, semplicemente perché il controllo sui permessi manca a livello di endpoint o di metodo HTTP.
6. **API6:2023 – Unrestricted Access to Sensitive Business Flows** — la singola richiesta può essere anche legittima, ma se ripetuta migliaia di volte automatizza un abuso della logica di business (es. acquisto massivo di biglietti, creazione automatica di account).
7. **API7:2023 – Server Side Request Forgery (SSRF)** — l'API accetta un URL fornito dall'utente e lo interroga senza validarlo, permettendo di raggiungere risorse interne altrimenti non accessibili dall'esterno.
8. **API8:2023 – Security Misconfiguration** — header di sicurezza mancanti, CORS³ troppo permissivo, ambienti di debug ancora attivi in produzione: stessa famiglia di errori vista nel capitolo precedente, qui applicata allo strato applicativo.
9. **API9:2023 – Improper Inventory Management** — endpoint vecchi, versioni deprecate o ambienti di staging dimenticati online, mai rimossi né monitorati, restano comunque raggiungibili da chi li trova.

³CORS (*Cross-Origin Resource Sharing*): il meccanismo con cui un server dichiara quali siti di origine diversa possono leggere le sue risposte dal browser. Una politica permissiva le rende leggibili a qualunque origine.

10. **API10:2023 – Unsafe Consumption of APIs** — la vulnerabilità non nasce nell'API stessa, ma nella fiducia cieca riposta nei dati restituiti da API di terze parti integrate nel sistema.

Tra le dieci categorie appena elencate la prima merita un'attenzione diversa dalle altre: non tanto per la posizione in classifica, quanto per ciò che rappresenta, cioè un controllo di accesso che manca proprio dove ci si aspetterebbe fosse più solido, sulla singola risorsa richiesta. È il tipo di errore che si insinua facilmente in un endpoint scritto in fretta e che, allo stesso tempo, può esporre porzioni estese della base dati di un sistema con una richiesta HTTP modificata di poco. La sottosezione seguente lo esamina nel dettaglio: come si manifesta, da quali cause nasce e con quali misure lo si previene.

2.2.1 API1:2023 – Broken Object Level Authorization

Definizione e collocazione La Broken Object Level Authorization (BOLA) si manifesta quando un'applicazione, pur autenticando correttamente l'utente che effettua una richiesta, non verifica se quello stesso utente abbia effettivamente il diritto di accedere all'oggetto specifico richiesto [5].

Un'interfaccia di programmazione applicativa espone tipicamente un insieme di risorse — utenti, ordini, documenti, transazioni — identificabili singolarmente attraverso un riferimento univoco. L'accesso a tali risorse presuppone due verifiche concettualmente distinte, spesso confuse nella pratica implementativa: l'autenticazione, che accerta l'identità del soggetto che effettua la richiesta, e l'autorizzazione, che stabilisce se quel soggetto, una volta identificato, abbia il diritto di operare su una specifica risorsa.

Un utente autenticato correttamente non è, per definizione, un utente autorizzato ad accedere a qualunque oggetto gestito dal sistema. Questa distinzione, apparentemente elementare, è proprio il punto in cui molte implementazioni falliscono: il possesso di credenziali valide viene talvolta considerato — implicitamente, nel disegno del codice — condizione sufficiente per l'accesso a una risorsa, quando in realtà rappresenta soltanto la premessa per una verifica ulteriore, relativa alla relazione specifica tra quell'utente e quell'oggetto.

Funzionamento delle API e ruolo degli identificatori di risorsa Le API REST, e in misura diversa quelle basate su GraphQL⁴, espongono le risorse attraverso identificatori che compaiono nel percorso della richiesta, nei parametri di query, nel corpo del messaggio o negli header. Un endpoint come `GET /api/orders/482` restituisce, nella maggior parte delle implementazioni, l'oggetto ordine identificato dal valore 482, indipendentemente dal fatto che tale valore sia un intero progressivo, un UUID o un altro tipo di riferimento.

La gestione di una richiesta di questo tipo comporta, lato server, due passaggi distinti: la risoluzione dell'identificatore in un oggetto concreto, tramite interrogazione del database o di un servizio dati, e il controllo che l'utente autenticato abbia diritto di accesso a quell'oggetto specifico. Il primo passaggio è quasi sempre implementato correttamente, poiché è indispensabile al funzionamento stesso dell'endpoint; il secondo, non avendo un effetto visibile quando manca, viene più facilmente trascurato o dato per scontato in fase di sviluppo.

Va osservato che l'endpoint, in questo scenario, non è di per sé vietato all'utente: la sua configurazione di ruoli e permessi lo autorizza a invocare quella funzione API. Questo distingue concettualmente BOLA da un'altra categoria della stessa famiglia, la Broken Function Level Authorization (BFLA), in cui è l'accesso alla funzione stessa a non essere autorizzato. In BOLA la funzione è legittima; il problema riguarda esclusivamente quale istanza dell'oggetto venga restituita o modificata.

Meccanismo dell'attacco Lo sfruttamento di una vulnerabilità BOLA non richiede, nella maggior parte dei casi, strumenti sofisticati né la compromissione di credenziali altrui. È sufficiente disporre di un account legittimo sul sistema bersaglio. Il percorso tipico dell'attacco segue una sequenza riconoscibile: l'utente accede al proprio account, richiede una propria risorsa attraverso l'interfaccia normale dell'applicazione e osserva, tramite gli strumenti di sviluppo del browser o un proxy di intercettazione, l'identificatore associato a quella risorsa nella richiesta sottostante.

⁴GraphQL: linguaggio di interrogazione per API in cui il client, anziché richiamare endpoint predefiniti, descrive in una sola richiesta l'esatto insieme di dati che desidera ricevere.

A questo punto l'identificatore viene sostituito con un altro valore plausibile per lo stesso schema — un intero vicino, nel caso di sequenze numeriche, oppure un valore ottenuto per enumerazione più sistematica quando lo schema lo consente — e la richiesta viene reinviata con lo stesso token di autenticazione, valido e non modificato. Ad esempio:

```
GET /api/orders/483 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_A>
```

dove 483 non è l'ordine appartenente al titolare del token, ma un identificatore vicino a quello osservato in precedenza durante il normale utilizzo dell'applicazione. La richiesta è, dal punto di vista del protocollo, del tutto regolare: cambia soltanto il valore del parametro. Che la risposta contenga davvero i dati di un altro utente dipende da un unico fattore, ripreso nel paragrafo seguente: se il server confronti o meno l'identità del chiamante con il proprietario effettivo dell'oggetto.

Esempio applicativo Si consideri la risorsa `/api/orders/{id}`, utilizzata per recuperare i dettagli di un ordine. Una richiesta legittima, effettuata dal proprietario dell'ordine 482, ha la forma:

```
GET /api/orders/482 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_A>
```

e produce una risposta simile alla seguente:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "order_id": 482,
  "customer_id": 1041,
  "items": [...],
```

```
"shipping_address": "..."  
}
```

La richiesta modificata introdotta nel paragrafo precedente, diretta all'ordine 483 e appartenente a `customer_id` 1078, viene gestita da un'implementazione vulnerabile allo stesso modo: l'identificatore viene estratto dal percorso, la tabella corrispondente viene interrogata, il risultato viene restituito. In nessun punto di questo flusso il valore di `customer_id` restituito viene confrontato con l'identità associata al token presente nell'header `Authorization`. La risposta, di conseguenza, è ancora un 200 OK, questa volta con i dati di un ordine che non appartiene al chiamante.

Il punto critico non è la struttura della richiesta, identica nei due casi, ma l'assenza di un confronto esplicito fra utente autenticato e risorsa richiesta. Un'implementazione corretta dovrebbe, prima di restituire l'oggetto, verificare che il `customer_id` dell'ordine coincida con l'identità del chiamante — o che quest'ultimo disponga di un diritto di accesso esplicitamente concesso su quella risorsa — e restituire, in caso contrario, un errore di autorizzazione: tipicamente 403 `Forbidden`, oppure 404 `Not Found` quando si preferisce non rivelare l'esistenza della risorsa a chi non ne ha diritto.

Cause e meccanismi tecnici tipici L'assenza di un controllo di autorizzazione a livello di oggetto è la causa immediata di ogni istanza di BOLA, ma raramente costituisce una scelta progettuale deliberata. Più spesso deriva da alcune condizioni che meritano di essere analizzate singolarmente.

La prima riguarda la sovrapposizione concettuale tra autenticazione e autorizzazione nella fase di implementazione. Un endpoint protetto da un middleware di autenticazione, che verifica la validità del token e ne estrae l'identità, può apparire sicuro a chi lo sviluppa, inducendo a trascurare la necessità di un controllo ulteriore specifico per l'oggetto acceduto. Il middleware risponde alla domanda "chi sta chiamando?", non a "questo chiamante ha diritto su questo oggetto?".

Una seconda causa è la fiducia implicita nei parametri forniti dal client. Alcune implementazioni derivano l'identità del proprietario di una risorsa dallo stesso identificatore fornito nella richiesta, oppure presuppongono che, poiché l'interfaccia utente dell'applicazione non espone mai identificatori altrui, un client "onesto"

non li invierebbe mai. Questa assunzione ignora che un client HTTP può essere costruito o modificato arbitrariamente da chi effettua la richiesta, indipendentemente dall'interfaccia grafica ufficiale.

Un terzo fattore è rappresentato da controlli di autorizzazione implementati in modo incompleto o incoerente tra i diversi endpoint di una stessa API. È relativamente comune riscontrare verifiche corrette sull'operazione di cancellazione di una risorsa e verifiche assenti sull'operazione di lettura o aggiornamento della medesima risorsa, poiché ciascun endpoint viene sviluppato e sottoposto a revisione in momenti diversi, con criteri non sempre uniformi. La crescita organica di un'API nel tempo, con l'aggiunta progressiva di nuovi endpoint da parte di sviluppatori diversi, tende ad amplificare questa disomogeneità.

Una causa più strutturale riguarda infine la gestione delle relazioni tra utenti e risorse nel modello dati. Quando tale relazione non è rappresentata in modo esplicito e interrogabile — ad esempio tramite una chiave esterna diretta tra la tabella degli utenti e quella delle risorse, o tramite un livello di autorizzazione centralizzato — ogni endpoint deve reimplementare autonomamente la logica di verifica, e questo aumenta la probabilità che qualche percorso venga trascurato.

Impatto L'impatto di una vulnerabilità BOLA dipende dalla natura della risorsa esposta, ma incide tipicamente su più proprietà di sicurezza contemporaneamente. Sul piano della riservatezza, l'accesso non autorizzato a un oggetto consente la lettura di dati che dovrebbero rimanere riservati al proprietario legittimo: informazioni personali, dettagli finanziari, contenuti sanitari, comunicazioni private, a seconda del dominio applicativo.

Quando l'endpoint vulnerabile consente anche operazioni di scrittura, l'impatto si estende all'integrità delle risorse: un attaccante può modificare dati appartenenti ad altri utenti, alterare lo stato di un ordine, cambiare le impostazioni di un account che non gli appartiene. Se l'operazione esposta è una cancellazione, la conseguenza diventa la perdita di disponibilità della risorsa per il legittimo proprietario.

Un aspetto che distingue BOLA da molte altre classi di vulnerabilità è la sua natura sistematicamente ripetibile: una volta individuato un endpoint privo del controllo appropriato, lo stesso schema di attacco può essere applicato a un numero

elevato di identificatori, trasformando quella che sembra una singola falla puntuale in un canale di accesso a porzioni estese della base dati.

Il rischio assume connotati particolari negli ambienti multi-tenant e cloud, dove più clienti condividono la stessa infrastruttura applicativa e la separazione tra i rispettivi dati dipende interamente dalla correttezza della logica di autorizzazione, non da una separazione fisica delle risorse. In un simile contesto, una singola vulnerabilità BOLA non compromette soltanto i dati di un utente, ma può erodere il confine logico tra tenant distinti, con implicazioni che vanno oltre il singolo incidente e toccano la fiducia complessiva riposta nell'infrastruttura condivisa.

Mitigazione La prevenzione di BOLA richiede che ogni accesso a una risorsa identificata da un parametro fornito dal client sia accompagnato da una verifica esplicita del diritto dell'utente su quello specifico oggetto, eseguita interamente lato server. Nessuna informazione proveniente dal client, incluso l'identificatore stesso della risorsa, può essere considerata prova di autorizzazione: l'identificatore indica quale oggetto viene richiesto, non chi ha diritto di accedervi.

Un principio applicabile in modo trasversale è quello del *least privilege*, secondo cui un soggetto dovrebbe disporre soltanto dei permessi strettamente necessari a svolgere le proprie funzioni, senza margini di accesso più ampi per comodità implementativa. Applicato al livello oggetto, questo si traduce nella necessità di associare esplicitamente ogni utente, o ruolo, alle risorse su cui ha effettivamente diritto di operare, evitando che l'accesso derivi da assunzioni implicite sulla struttura delle richieste.

Dal punto di vista architetturale, è preferibile centralizzare la logica di autorizzazione in un livello comune — un modulo, un middleware o un servizio dedicato — piuttosto che reimplementarla in modo indipendente in ciascun endpoint. Questa scelta non elimina la necessità di applicare il controllo in ogni punto di accesso, ma riduce la probabilità che errori o dimenticanze locali producano percorsi non protetti, e semplifica la verifica di coerenza tra le diverse funzioni esposte dall'API.

La coerenza dei controlli tra operazioni di lettura, scrittura e cancellazione sulla stessa risorsa merita infine attenzione specifica: la protezione di un'operazione non

implica automaticamente la protezione delle altre, ed è in questa asimmetria che si annida buona parte delle istanze reali di BOLA.

Le sottosezioni seguenti trattano in forma più sintetica le restanti categorie della OWASP API Security Top 10, a partire da API2:2023.

2.2.2 API2:2023 – Broken Authentication

Definizione e collocazione Broken Authentication occupa la seconda posizione della classificazione OWASP e si colloca concettualmente a monte di BOLA: mentre quest'ultima riguarda il controllo sull'accesso a uno specifico dato o risorsa, Broken Authentication compromette la capacità stessa del sistema di stabilire con certezza l'identità di chi effettua la richiesta. Un fallimento in questa fase rende vano qualsiasi controllo successivo, poiché un meccanismo di autorizzazione perde ogni efficacia se l'identità su cui si fonda può essere falsificata o aggirata [6]. A differenza di altre categorie più circoscritte, questa voce raggruppa un insieme eterogeneo di debolezze nei processi di verifica dell'identità: dalla gestione poco attenta dei token di sessione all'assenza di vincoli sul numero di tentativi di accesso consentiti. Un esempio comune riguarda i messaggi di risposta del server: se il sistema indica esplicitamente quale dei due campi, nome utente o password, sia errato, fornisce involontariamente all'attaccante una conferma sull'esistenza di quell'account. Parallelamente, l'assenza di limitazioni sulla frequenza delle richieste lascia l'interfaccia di login esposta ad attacchi automatizzati volti a indovinare le credenziali su larga scala.

Cause e meccanismi tecnici tipici Spesso queste problematiche nascono dal tentativo di implementare soluzioni di autenticazione proprietarie o non standard. Come evidenziato dalla documentazione OWASP, strumenti come le API key o protocolli di autorizzazione come OAuth vengono talvolta impiegati impropriamente per identificare gli utenti in scenari per cui non sono stati progettati, creando falle strutturali [6]. Un ulteriore fattore critico è la mancanza di controlli sul volume dei tentativi di accesso. Endpoint di autenticazione e flussi di recupero password privi di rate limiting risultano vulnerabili ad attacchi in cui elenchi di credenziali già compromesse altrove vengono provati in massa, sfruttando la tendenza diffusa a riutilizzare le medesime password su più servizi [7]. Infine, la gestione dei token di sessione (come i JSON Web Token, o JWT) introduce ulteriori insidie: errori nella convalida della firma crittografica, chiavi di cifratura deboli, mancato controllo dei tempi di scadenza o assenza di meccanismi di revoca immediata permettono a un attaccante di forgiare token validi o di continuare a utilizzare credenziali che dovrebbero essere

già decadute [8].

Impatto L'impatto di Broken Authentication è per sua natura sistemico: assumendo il controllo dell'identità di un utente, l'attaccante ne eredita tutti i privilegi operativi senza che il backend possa distinguere le azioni legittime da quelle malevole [6]. Ciò può tradursi non solo nell'accesso a dati riservati, ma anche nella manomissione dei parametri di sicurezza dell'account (come la modifica dell'indirizzo di recupero), trasformando un'intrusione temporanea nella perdita definitiva del profilo.

Mitigazione La contromisura principale consiste nell'affidarsi a standard industriali consolidati e a librerie verificate, evitando soluzioni "fai-da-te" per la gestione di token e password [9]. A livello applicativo e infrastrutturale è indispensabile introdurre politiche di rate limiting dedicate e restrittive sugli endpoint di login, adottare l'autenticazione a più fattori (MFA) dove possibile e imporre una nuova conferma delle credenziali prima di eseguire modifiche critiche sul profilo utente [10].

2.2.3 API3:2023 – Broken Object Property Level Authorization

Definizione e collocazione La *Broken Object Property Level Authorization* (BOPLA) identifica una classe di vulnerabilità nella quale i controlli di autorizzazione applicati alle API non considerano adeguatamente le singole proprietà degli oggetti gestiti dall'applicazione. Il problema non riguarda quindi soltanto la possibilità per un utente di accedere a un determinato oggetto, ma anche quali attributi di quell'oggetto possa effettivamente leggere o modificare [4].

La distinzione rispetto alla *Broken Object Level Authorization* (BOLA) è rilevante. Nel caso di BOLA, il controllo di autorizzazione fallisce a livello dell'oggetto: un utente riesce, per esempio, ad accedere a una risorsa appartenente a un altro utente manipolando l'identificativo della risorsa. Nel caso di BOPLA, invece, l'utente può avere legittimamente accesso all'oggetto, ma ottenere o modificare proprietà che non rientrano nei suoi privilegi. L'oggetto, quindi, non costituisce necessariamente la risorsa a cui viene negato l'accesso; è una sua specifica proprietà a essere esposta o manipolabile senza un'adeguata autorizzazione.

Il rischio è particolarmente rilevante nelle API REST, dove è frequente che un endpoint restituisca direttamente la rappresentazione serializzata di un oggetto. Una struttura interna composta da numerosi attributi può così diventare, in assenza di una selezione esplicita, il corpo stesso della risposta inviata al client.

Cause e meccanismi tecnici tipici Una delle cause più comuni è la sovrapposizione tra il modello utilizzato internamente dall'applicazione e quello esposto attraverso l'API. Un oggetto memorizzato nel database può contenere, ad esempio, proprietà quali `id`, `name`, `email`, `role` e `internalStatus`. Non tutte queste proprietà hanno però la stessa finalità né devono necessariamente essere accessibili allo stesso insieme di utenti.

Un'implementazione che serializza automaticamente l'intero oggetto può produrre una risposta contenente anche attributi che il client non dovrebbe conoscere. In questo caso il problema non deriva necessariamente da un accesso non autorizzato all'oggetto, ma dal fatto che l'API non applica una politica distinta alle proprietà che lo compongono. L'interfaccia grafica potrebbe nascondere determinati campi, ma ciò non costituisce una misura di sicurezza: il contenuto della risposta HTTP rimane osservabile dal client e può essere analizzato direttamente.

Un secondo meccanismo riguarda le operazioni di scrittura. Framework e librerie possono consentire il *binding* automatico dei parametri ricevuti dal client verso gli attributi di un oggetto applicativo. Se il server accetta un payload come:

```
{  
  "name": "Mario",  
  "email": "mario@example.com"  
}
```

un attaccante può provare ad aggiungere proprietà che non erano previste dal normale flusso applicativo:

```
{  
  "name": "Mario",  
  "email": "mario@example.com",  
  "role": "ADMIN"
```

```
"role": "admin"  
}
```

Se il campo `role` viene accettato e aggiornato senza un controllo specifico, l'utente può modificare una proprietà interna che dovrebbe essere gestita esclusivamente dal sistema. Lo stesso principio può riguardare attributi quali `isAdmin`, `approved`, `price` o `ownerId`, a seconda del modello utilizzato dall'applicazione.

Il problema può quindi manifestarsi in due direzioni differenti: lettura non autorizzata di proprietà e modifica non autorizzata di proprietà. Nel primo caso l'attaccante osserva informazioni che non dovrebbe ricevere; nel secondo riesce a influenzare direttamente lo stato dell'applicazione attraverso attributi che non dovrebbero essere controllabili dal client.

Impatto L'impatto di una vulnerabilità BOPLA dipende principalmente dal tipo di proprietà coinvolta e dalle funzioni che la utilizzano. La semplice esposizione di un attributo può comportare una violazione della riservatezza, mentre la possibilità di modificarlo può compromettere l'integrità dei dati o il funzionamento dell'applicazione.

Nel caso di proprietà contenenti informazioni personali, finanziarie o relative all'organizzazione interna del sistema, l'esposizione può determinare una divulgazione di dati a soggetti non autorizzati. La possibilità di modificare determinate proprietà può invece portare alla perdita o alla corruzione dei dati. In situazioni più critiche, una proprietà utilizzata nei meccanismi di autorizzazione o nella gestione dello stato dell'account può essere sfruttata per ottenere un'escalation dei privilegi o contribuire alla compromissione dell'account. La gravità della vulnerabilità dipende quindi dal significato che quella proprietà assume all'interno dell'applicazione, più che dal meccanismo che ne consente l'accesso.

Mitigazione La mitigazione di BOPLA richiede che l'autorizzazione venga applicata non soltanto all'endpoint o all'oggetto, ma anche alle proprietà che possono essere restituite o modificate. La prima misura consiste quindi nel definire esplicitamente quali attributi ogni endpoint può esporre e quali possono essere accettati in input.

Per le risposte è preferibile evitare i meccanismi di serializzazione automatica, che restituiscono l'oggetto così com'è rappresentato internamente, e selezionare invece le proprietà necessarie alla singola operazione.

Per le operazioni di scrittura è necessario limitare esplicitamente gli attributi modificabili dal client. Una tecnica comune consiste nell'utilizzare una *whitelist* delle proprietà ammesse, evitando che il payload venga associato automaticamente a tutti gli attributi dell'oggetto.

2.2.4 API4:2023 – Unrestricted Resource Consumption

Ogni richiesta servita da un'API consuma risorse: banda, CPU, memoria, storage e, con frequenza crescente, servizi a pagamento di terze parti. API4:2023 raccoglie i casi in cui almeno uno dei limiti che dovrebbero governare quel consumo è assente o dimensionato male, fra cui i timeout di esecuzione, la memoria allocabile e il numero di processi [11]. Non è il difetto di una funzione specifica, ma una proprietà trasversale di qualunque endpoint: i limiti di consumo sono la difesa in profondità di altre categorie, e la loro assenza si intreccia con gli endpoint di autenticazione di API2, con l'automazione dei flussi di business di API6, con la copertura incompleta della superficie esposta di API9 e con le chiamate verso servizi esterni di API10.

La radice comune è un'asimmetria: produrre una richiesta costa al client quasi nulla, servirla può costare al server una quantità che cresce in funzione di parametri che il client controlla, e finché quella funzione di costo non ha un limite superiore l'API resta esposta. I meccanismi sono tre. Il volume puro, dove più insidiosa dell'assenza del limite è la sua presenza in una sola dimensione, perché un tetto per indirizzo IP cade contro un attacco distribuito e uno per endpoint non intercetta il batching. La moltiplicazione di operazioni dentro una richiesta sola, illustrata dallo scenario GraphQL in cui centinaia di mutazioni aggregate in un unico batch saturano la memoria aggirando sia il limite di frequenza sia quello sulla dimensione delle immagini [11, 12]. Infine il costo algoritmico della singola richiesta [13], con il ReDoS come istanza più diffusa, documentata nei server JavaScript dove il thread singolo rende una richiesta sufficiente a congelare il processo [14]. Vi si aggiunge la causa che l'edizione 2023 porta in primo piano, l'assenza di soglie di spesa verso

i fornitori esterni, dove la risorsa non è computazionale ma economica. Che limiti tanto semplici manchino dipende da due fatti: possono risiedere nel framework, nel proxy, nel gateway⁵ o presso il fornitore cloud, e quando ogni livello presume che se ne occupi un altro non se ne occupa nessuno; e la loro assenza non produce errori, sicché un endpoint privo di limiti supera ogni test funzionale.

L'esito immediato è il denial of service per esaurimento, che CWE-770⁶ enumera come conseguenza principale sulla disponibilità [15], ma il punto di rottura è la logica applicativa anziché la banda: una sola richiesta ReDoS⁷ o un solo batch GraphQL bastano, e nessuna difesa perimetrale basata sul volume li riconosce come anomali.

Nella mitigazione nessun livello basta da solo: i limiti fisici su memoria, CPU e processi sono dichiarati alla piattaforma, mentre all'ingresso dei dati va fissata una dimensione massima per ogni parametro e payload, vincolo che il contratto OpenAPI permette di dichiarare esplicitamente. Il rate limiting, segnalato al client con il codice 429 [16, 17], va tarato per endpoint e affiancato da un limite sulle esecuzioni della singola operazione per utente, che il conteggio delle chiamate non cattura [11], e si irrigidisce sugli endpoint di autenticazione [6, 9]; NIST lo colloca nel gateway anziché nel singolo servizio [18], scelta corretta purché ogni host esposto stia dietro quel gateway. Per la famiglia algoritmica servono timeout e motori a tempo lineare [14], per GraphQL l'analisi statica del costo di una query prima di eseguirla [12], per la dimensione economica soglie di spesa o allarmi di fatturazione.

2.2.5 API5:2023 – Broken Function Level Authorization

Un'API espone operazioni, e non tutte sono destinate a tutti. Creare un invito con privilegi amministrativi, esportare l'elenco degli utenti, cancellare un account altrui sono funzioni riservate a un ruolo, e il controllo che verifica quel ruolo prima

⁵*Gateway*: componente posto davanti ai servizi applicativi che riceve tutte le chiamate esterne e vi applica funzioni comuni quali autenticazione, instradamento e limiti di frequenza.

⁶CWE (*Common Weakness Enumeration*): il catalogo pubblico delle debolezze software mantenuto da MITRE; ogni voce, indicata dalla sigla CWE seguita da un numero, descrive una classe di difetto indipendentemente dal prodotto in cui compare.

⁷ReDoS (*Regular expression Denial of Service*): un input costruito ad arte che fa degenerare il tempo di valutazione di un'espressione regolare, saturando la CPU con una richiesta sola.

di eseguirle è l'autorizzazione a livello di funzione: API5:2023 raccoglie i casi in cui tale controllo manca o è aggirabile, e un utente ordinario riesce a invocare funzioni riservate [19]. La differenza rispetto ad API1:2023 è di livello, ed è l'unica che OWASP fissi in modo prescrittivo: nella BOLA l'accesso all'endpoint è previsto e la violazione avviene manipolando l'identificatore dell'oggetto, mentre se l'attaccante raggiunge una funzione cui non dovrebbe accedere il caso ricade sotto la BFLA [5]. Il riferimento tassonomico è CWE-285 *Improper Authorization*, che MITRE sconsiglia però di usare per mappare vulnerabilità reali, rimandando alle voci figlie CWE-862 quando il controllo è assente e CWE-863 quando esiste ma decide male [20].

Il controllo funzionale manca perché non ha un luogo obbligato: OWASP osserva che l'autorizzazione su una funzione è gestita tramite configurazione oppure a livello di codice, e che scrivere controlli corretti è confuso, date le molte gerarchie di ruoli e gruppi delle applicazioni moderne [19]. Vi si somma il valore predefinito, perché una rotta aggiunta a un controller eredita il routing e l'autenticazione, non l'autorizzazione: nasce raggiungibile e tale resta. Sfruttarla non richiede manipolazioni, bastano chiamate legittime verso un endpoint cui non si dovrebbe accedere, e la struttura prevedibile delle API rende il tentativo facile, sostituendo GET con PUT oppure users con admins: la stessa regolarità che rende un'API documentabile la rende enumerabile. Lo scenario di riferimento è un'applicazione ad accesso su invito, in cui l'app mobile recupera un invito già emesso con una chiamata del tutto ordinaria:

```
GET /api/invites/{invite_guid} HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_normale>
```

La risposta espone, fra gli altri campi, il ruolo associato all'invito. L'attaccante ne cambia allora metodo ed endpoint, riutilizzando lo stesso token e senza manipolare alcun identificatore:

```
POST /api/invites/new HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_normale>
```

```
Content-Type: application/json
```

```
{"email": "attaccante@example.com", "role": "admin"}
```

L'operazione è riservata agli amministratori, ma soltanto attraverso una console priva di controllo di funzione: il server la esegue [19]. Il controllo, in un certo senso, esisteva, ma stava nella console, nel fatto che solo un amministratore vedesse quel pulsante. Tre osservazioni spostano infine il difetto fuori dal singolo gestore: il percorso non dice se un endpoint sia amministrativo, perché le funzioni riservate stanno spesso sotto `api/users` anziché sotto `api/admins` [19]; i verbi HTTP lasciati abilitati per inerzia riaprono l'operazione da una via non presidiata [21]; le versioni obsolete continuano a servire il gestore precedente al controllo [22].

OWASP colloca l'impatto tecnico nella fascia grave, con esiti che vanno dalla divulgazione alla perdita o corruzione dei dati, fino all'interruzione del servizio [19]. Il danno non scala però con il numero di identificatori, bensì con la potenza della funzione raggiunta: una BOLA restituisce un record per richiesta, mentre una BFLA su un endpoint di esportazione ottiene l'intera base dati con una chiamata sola; e poiché le funzioni riservate sono in prevalenza funzioni di scrittura, il difetto che espone l'esportazione degli utenti espone, sull'endpoint accanto, la loro cancellazione.

La prescrizione centrale riguarda il valore predefinito: negare ogni accesso e richiedere concessioni esplicite ai ruoli per ogni funzione [19], con un vantaggio che sta in come si manifesta l'errore, perché se il default è il diniego una dimenticanza produce un 403 che il primo test funzionale rivela, mentre se il default è il permesso la stessa dimenticanza non produce alcun segnale. Segue la revisione degli endpoint alla luce della logica di business e della gerarchia dei gruppi, lavoro che non ammette scorciatoie rispetto alla conoscenza del dominio. La terza è l'unica con una forma sintattica: i controller amministrativi devono ereditare da un controller astratto che applichi il controllo in base a ruolo o gruppo, misura che non aggiunge verifiche ma le riposiziona, perché ometterle richiede l'atto deliberato di collocare un gestore fuori dalla gerarchia, discontinuità visibile in revisione. Nessuna delle tre risolve però la copertura, perché il controllo protegge solo la superficie servita dal codice che lo contiene [22, 21].

2.2.6 API6:2023 – Unrestricted Access to Sensitive Business Flows

Esistono attacchi che non violano alcuna regola. Qui ogni singola chiamata è legittima: l'utente ha diritto di compierla, i controlli di autorizzazione funzionano, il codice si comporta come progettato, e il danno nasce dal numero. OWASP definisce vulnerabile l'endpoint che espone un flusso di business sensibile senza limitarne adeguatamente l'accesso, portando come esempi l'acquisto di un prodotto, la creazione di un commento e la prenotazione di una fascia oraria [23]. Segue l'affermazione che definisce davvero la categoria: il rischio di un accesso eccessivo cambia da settore a settore, e la creazione di post tramite script può costituire spam per un social network ed essere incoraggiata da un altro [23]. Nessuna proprietà del codice, del contratto o della configurazione rende dunque un endpoint vulnerabile ad API6: lo stesso identico endpoint è sicuro in un contesto e vulnerabile in un altro, e a decidere è il modello di business che gli sta dietro.

Coerentemente, la causa che OWASP indica non è tecnica: manca una visione d'insieme dell'API che ne supporti i requisiti di business [23]. Il difetto nasce nel raccordo fra chi progetta l'endpoint e chi conosce il valore economico di ciò che l'endpoint muove, e dove quel raccordo manca nessuno è nella posizione di accorgersene. Lo sfruttamento richiede di comprendere il modello di business, individuare i flussi sensibili e automatizzarne l'accesso, e soltanto l'ultimo passo produce traffico osservabile: il primo scenario lo illustra con una console da gioco annunciata in quantità limitata, acquistata da un programma eseguito il giorno del lancio e distribuita su indirizzi IP e località diverse [23], dove la distribuzione geografica non è un dettaglio di colore ma la contromisura dell'attaccante contro il limite per indirizzo, e spiega perché il rate limiting nella sua forma più semplice non basti.

Sull'impatto il documento OWASP è internamente incoerente: la tabella dei punteggi classifica l'impatto tecnico come moderato, mentre la descrizione degli impatti afferma che un impatto tecnico in generale non è atteso e porta come esempi di danno l'impossibilità per gli utenti legittimi di acquistare un prodotto e l'inflazione nell'economia interna di un videogioco [23]. È la seconda formulazione a corrispondere agli scenari descritti, che non producono né violazione di riservatezza né alterazione di dati né indisponibilità del servizio: ciò che cambia sta fuori dal sistema e si misura in

scorte sottratte, ricavi persi o costi di acquisizione pagati per utenti inesistenti. Ne segue un effetto collaterale rilevante, perché un incidente che non produce indisponibilità né perdita di dati non attiva le procedure di risposta e viene percepito come problema commerciale anziché di sicurezza.

La mitigazione proposta ha una struttura a due livelli in cui l'ordine conta: prima si identificano, sul piano del business, i flussi che un uso eccessivo può danneggiare, e solo dopo si sceglie il meccanismo tecnico adeguato al rischio individuato [23]. Nessuna categoria precedente prescrive una fase preliminare simile, perché in nessuna altra la configurazione corretta dipende dall'azienda che la adotta. I meccanismi elencati, dal fingerprinting del dispositivo al captcha, dall'individuazione di comportamenti non umani come il passaggio dal carrello all'acquisto in meno di un secondo fino al blocco dei nodi di uscita Tor, sono presentati come modi per rallentare le minacce automatizzate, non per impedirle, e la logica che li accomuna è economica: nessuno produce una decisione binaria affidabile, tutti spostano il costo dell'attacco fino a superare il profitto atteso.

2.2.7 API7:2023 – Server Side Request Forgery

Il traffico non viaggia in una sola direzione: verso servizi interni, archivi remoti e risorse indicate da chi interroga il servizio partono richieste che l'applicazione stessa emette, ed è su queste che agisce l'attaccante in API7:2023.

La causa immediata è l'assenza o l'inadeguatezza della validazione dell'URI fornito dal client, e sono i modelli di sviluppo moderni a incoraggiare l'accesso a URI di provenienza esterna: il difetto nasce quindi da una funzionalità voluta, perché nessuno introduce per sbaglio un endpoint che scarica un'immagine da un indirizzo scelto dall'utente. I due scenari del documento mostrano le due forme dell'attacco. Nella forma cieca un social network consente di indicare l'immagine del profilo tramite URL, e l'attaccante vi scrive un indirizzo interno:

```
POST /api/profile/upload_picture HTTP/1.1
Host: api.example.com
Content-Type: application/json
```

```
{"picture_url": "http://localhost:8080"}
```

Misurando il tempo di risposta deduce se la porta sia aperta, e scandisce così la rete interna [24]: l'informazione non arriva dal corpo della risposta, ma dalla sua latenza. Nella forma con ritorno un prodotto di sicurezza invia una richiesta di prova al webhook che l'utente sta configurando e gliene mostra la risposta, e l'attaccante vi indica l'indirizzo link-local del servizio di metadati dell'ambiente cloud — `http://169.254.169.254/latest/meta-data/` nella convenzione più diffusa — ottenendone le credenziali [24]: il webhook rende comune il difetto, l'indirizzo a percorso noto lo rende grave. Gli esiti vanno dall'enumerazione dei servizi interni per scansione di porte alla divulgazione di informazioni, dall'aggiramento di firewall all'uso del server come proxy [24]. L'elemento che distingue l'SSRF è il cambio di perimetro: l'attaccante non ottiene un dato, ottiene una posizione, quella del server, e il valore dell'attacco dipende perciò da cosa sia raggiungibile dall'interno. I servizi di metadati cloud convertono poi l'SSRF da ricognizione a compromissione, perché rispondono a un indirizzo noto senza autenticazione ed espongono le credenziali del ruolo associato alla macchina [25].

Sul piano della rete OWASP raccomanda di isolare il meccanismo di recupero delle risorse, che di norma mira a risorse remote e non interne: è la difesa più robusta perché non dipende dalla correttezza della validazione, dato che se il componente che effettua le richieste in uscita non ha rotta verso la rete interna un URL malevolo non produce nulla. Sul piano applicativo sono prescritte liste di elementi ammessi su tre dimensioni — origini remote attese, schemi di URL e porte, tipi di media accettati — ciascuna delle quali blocca una famiglia di aggiramenti diversa, cui seguono la disabilitazione delle redirezioni, un parser di URL collaudato e il divieto di restituire al client le risposte grezze [24]: quest'ultima non impedisce l'attacco, ma ne degrada l'esito dalla forma con ritorno a quella cieca. Va infine registrata un'ammissione che nessun'altra categoria contiene: il rischio SSRF non può sempre essere eliminato del tutto [24], perché funzionalità e vulnerabilità coincidono.

2.2.8 API8:2023 – Security Misconfiguration

Un controllo che manca, un limite non impostato, un URL non validato: ciascuna delle categorie precedenti indica un punto preciso in cui il difetto risiede. La Security Misconfiguration non ha un punto: OWASP dichiara che la configurazione insicura può verificarsi a qualunque livello dello stack, dalla rete all'applicazione, e i suoi otto indicatori attraversano tutti quei livelli, dall'*hardening*⁸ assente ai verbi HTTP lasciati abilitati, dal TLS⁹ mancante ai messaggi di errore che espongono tracce dello stack [21]. Hanno una sola cosa in comune: nessuno è un errore di logica applicativa, perché il codice fa quello che deve ed è l'ambiente in cui gira a produrre l'esposizione. Lo conferma la particolarità tassonomica: fra le sette voci CWE associate alla categoria compare CWE-16 *Configuration*, di cui MITRE dichiara proibita la mappatura, invitando ad analizzare piuttosto il comportamento che la configurazione ha determinato [26]. Se ne ricava il criterio: la configurazione non è una debolezza, è il meccanismo attraverso cui una debolezza viene introdotta, sicché una politica CORS permissiva non è insicura in astratto, lo diventa perché produce un difetto di controllo di accesso.

La causa strutturale è che la configurazione non risiede in un luogo unico né sotto una responsabilità unica: si distribuisce fra codice, file di orchestrazione, gateway, impostazioni cloud e valori predefiniti delle librerie, ciascuno con un ciclo di vita proprio, e un valore corretto al rilascio può cessare di esserlo dopo un aggiornamento che nessuno ha collegato alla sicurezza. Il primo scenario mostra il predefinito pericoloso, ed è riconoscibilmente il caso Log4Shell: un'utilità di logging supporta le ricerche JNDI abilitate per impostazione predefinita, e quando espande un'intestazione contenente un riferimento JNDI a un server LDAP controllato dall'attaccante ne scarica ed esegue l'oggetto malevolo [21], senza che nella catena vi sia alcun difetto del codice applicativo. Il secondo mostra il meccanismo opposto, l'impostazione assente:

⁸*Hardening*: insieme delle operazioni con cui si riduce la superficie di attacco di un sistema: disattivazione di funzioni e servizi non necessari, rimozione delle credenziali predefinite, applicazione delle impostazioni consigliate.

⁹TLS (*Transport Layer Security*): il protocollo che cifra e autentica il canale di comunicazione su cui viaggia il traffico HTTP.

un social network espone i messaggi privati tramite `/dm/user_updates.json` e, non inviando l'intestazione `Cache-Control`, lascia che le conversazioni finiscano nella cache del browser, e qui non c'è nulla da disabilitare, c'è una direttiva che nessuno ha inviato. Resta poi la causa più diffusa, che l'elenco mette al secondo posto: patch mancanti e sistemi non aggiornati [21], difetto di manutenzione più che di configurazione.

Le configurazioni insicure, scrive OWASP, non solo espongono dati sensibili degli utenti, ma anche dettagli del sistema che possono condurre alla compromissione completa del server [21]: è la seconda parte a giustificare l'impatto severo, e i due scenari ne mostrano gli estremi, conversazioni private da un lato ed esecuzione di codice arbitrario dall'altro. Ciò che distingue la Security Misconfiguration è però la natura abilitante del difetto, che raramente costituisce l'attacco e quasi sempre la condizione che ne rende praticabile un altro: una traccia di stack non compromette nulla di per sé, ma rivela versioni e percorsi interni e abbassa il costo della ricognizione, mentre una politica di uscita permissiva trasforma un SSRF cieco in uno con ritorno. Vi si aggiunge l'ampiezza, perché un difetto di configurazione, essendo trasversale, non riguarda un endpoint ma tutti quelli serviti da quel componente.

Sul piano del processo OWASP chiede un hardening ripetibile, una revisione delle configurazioni sull'intero stack e un processo automatizzato che ne valuti l'efficacia in tutti gli ambienti, la cui implicazione comune conviene rendere esplicita: la configurazione va trattata come artefatto versionato e verificabile, non come stato accumulato di una macchina. Sul piano dei controlli le misure riguardano il canale cifrato verso ogni componente, i verbi HTTP da restringere, la politica CORS e le intestazioni di sicurezza, i tipi di contenuto accettati e gli schemi dei payload di risposta, comprese quelle di errore [21]; due meritano una nota, perché il canale cifrato è prescritto anche per le API interne, il che chiude l'assunto per cui il traffico interno sarebbe fidato, e lo schema dichiarato per le risposte di errore non è documentazione, ma un vincolo che a runtime impedisce la fuga di informazioni diagnostiche.

2.2.9 API9:2023 – Improper Inventory Management

Correggere un difetto presuppone sapere che esiste il componente che lo ospita, e fin qui quel componente è sempre stato noto: un handler, un limite, una configurazione. API9:2023 riguarda invece ciò che l'organizzazione non sa di avere: il difetto non è nel codice ma nella visibilità, e OWASP lo definisce come scarsa catalogazione degli host e delle versioni di un'API, articolandolo in due punti ciechi [22]. Il primo è quello della documentazione, perché restano senza risposta esplicite le domande su quale ambiente ospiti un'API, chi debba avervi accesso di rete e quale versione sia in esecuzione, e non esistono né un piano di dismissione né un inventario aggiornato degli host. Il secondo è quello dei flussi di dati, e ricorre quando dati sensibili raggiungono terze parti senza motivazione di business, approvazione o visibilità su cosa venga condiviso [22]. Anche qui il riferimento tassonomico è una voce non mappabile, CWE-1059 [27]: come per la Security Misconfiguration, la categoria non nomina un comportamento del prodotto, e nel suo caso non è un'imprecisione della tassonomia ma una proprietà del difetto, perché l'assenza di un inventario non è qualcosa che il software faccia.

La causa è un'asimmetria fra due costi: esporre una nuova istanza richiede pochi minuti, catalogarla e prevederne la dismissione richiede un processo che nessuna pipeline esegue da sola, e ogni ambiente di collaudo è però un'API completa e raggiungibile, mentre l'inventario che dovrebbe elencarlo è un documento aggiornato a mano. Il primo scenario ne mostra la conseguenza: un social network protegge il recupero della password dalla forza bruta, ma il meccanismo non è nel codice dell'API, bensì in un componente interposto fra il client e l'host ufficiale, e un ricercatore individua `beta.api.socialnetwork.owasp.org`, che esegue la stessa API senza esserne coperto, e reimposta la password di qualunque utente indovinando un token di sei cifre [22]: il controllo esisteva ed era corretto, ma non copriva tutti gli host che servivano quel codice. Il secondo riguarda l'altro punto cieco, con un'applicazione malevola che ottiene il consenso di 270.000 utenti ma, attraverso di essi, accede alle informazioni private di 50.000.000 di contatti: qui non c'è alcun host dimenticato, manca la conoscenza di dove i dati finiscano.

OWASP classifica l'impatto tecnico come moderato, ma il dettaglio decisivo è

che versioni diverse della stessa API sono talvolta collegate alla medesima base di dati [22]: l'host dimenticato non è una copia isolata, è una seconda porta sugli stessi dati. Ne discende il tratto distintivo, perché API9 non introduce una vulnerabilità propria ma rende raggiungibili quelle altrui e ne impedisce la correzione, dato che una patch si applica a ciò che si sa di avere in esecuzione; il giudizio di impatto moderato è quindi fuorviante, perché la gravità effettiva è quella del difetto peggiore che l'host non inventariato continua a servire.

Le contromisure muovono tutte dallo stesso presupposto, che la visibilità vada prodotta e non presupposta. Le prime sono di inventario — gli host, con ambiente, accesso di rete e versione, e i servizi integrati, con ruolo e sensibilità dei dati — cui segue la documentazione dell'API, che deve coprire autenticazione, errori, redirezioni, limitazione di frequenza, politica CORS ed endpoint. La misura più incisiva è però la successiva: generare la documentazione in modo automatico, con standard aperti e dentro la pipeline di integrazione continua [22], differenza non di grado ma di natura, perché trasforma la documentazione da documento parallelo, che diverge appena qualcuno dimentica di aggiornarlo, in artefatto prodotto dalla compilazione. Restano le prescrizioni sulle versioni: applicare le misure di sicurezza esterne a tutte quelle esposte e non alla sola produzione, che è ciò che il primo scenario mostra disatteso, e non usare dati di produzione fuori dalla produzione.

2.2.10 API10:2023 – Unsafe Consumption of APIs

Il perimetro di un servizio non coincide con il codice che lo compone: ogni dipendenza esterna è una superficie che non gli appartiene. API10:2023 rovescia perciò la prospettiva dell'attacco diretto, presupposta dalle nove categorie precedenti: l'attaccante non colpisce l'API bersaglio ma un servizio da cui essa dipende, e il danno arriva per un canale che l'applicazione considera fidato; le note di rilascio dichiarano di aver introdotto la categoria perché gli attaccanti hanno cominciato a compromettere i servizi integrati anziché le API del bersaglio [28]. La causa che OWASP individua è psicologica prima che tecnica: gli sviluppatori si fidano dei dati ricevuti da terze parti più di quanto si fidino dell'input dell'utente, e adottano standard più deboli quanto a trasporto, autenticazione e validazione [29]. Le cinque

condizioni di vulnerabilità sono canale non cifrato, dati non validati, redirezioni seguite ciecamente, risorse non limitate e timeout assenti.

La causa strutturale è un confine di fiducia collocato nel punto sbagliato. Un servizio di terze parti occupa una posizione ambigua: è esterno all'applicazione, ma scelto deliberatamente, raggiungibile su un canale che essa ha configurato e spesso offerto da un'azienda reputata affidabile, e la sua risposta attraversa così un confine che nessuno ha presidiato, perché nessuno lo ha riconosciuto come tale. Il primo scenario mostra l'iniezione differita: un'API si affida a un servizio esterno per arricchire gli indirizzi commerciali degli utenti e ne salva i dati in una base di dati SQL, e gli attaccanti vi memorizzano un payload di SQL injection associato a un'attività creata da loro, poi inducono l'API a prelevarla [29]; ciò che lo distingue da un'iniezione ordinaria è che l'attaccante non invia mai il payload all'API bersaglio, e sanificare l'input degli utenti non serve, perché quell'input è un innocuo nome di attività. Il secondo mostra la redirezione seguita ciecamente: un'API invia informazioni mediche a un fornitore su connessione cifrata, gli attaccanti lo compromettono ed esso risponde con un `308 Permanent Redirect`, sicché l'API ripete la richiesta identica, dati sensibili compresi, verso il loro server, e la fuga avviene per un comportamento predefinito del client HTTP che nessuno ha valutato come decisione di sicurezza. Il terzo scenario è il più istruttivo, perché la terza parte non deve nemmeno essere compromessa: un attaccante crea un repository Git denominato `' ; dropdb; --`, e l'applicazione che vi si integra ne usa il nome per costruire una query SQL ritenendolo sicuro [29], avendo confuso l'autenticità del canale con l'affidabilità del contenuto.

L'impatto varia a seconda di ciò che l'API bersaglio fa con i dati prelevati — esposizione di informazioni sensibili, molti tipi di iniezione o denial of service [29] — ed è classificato come severo. Ciò che caratterizza la categoria è che il difetto non ne determina l'entità, la quale è funzione di dove il dato non validato finisce: un'informazione prelevata da un fornitore e memorizzata può essere riletta mesi dopo da un componente che la tratta come dato interno, perché proviene dalla base di dati dell'organizzazione. Le contromisure prescritte sono quattro: valutare la postura di sicurezza dei fornitori al momento della scelta, imporre un canale di trasporto sicuro, validare e sanificare i dati ricevuti dalle API integrate, mantenere una lista delle

posizioni verso cui esse possono reindirizzare [29]. La terza è il nucleo della categoria ed è formulata negli stessi termini riservati all'input dell'utente: la contromisura consiste nell'abolire la distinzione fra le due sorgenti. Due delle cinque condizioni — timeout e limiti sulle risposte — restano però prive di contromisura corrispondente; quando il contratto del fornitore è noto, la risposta può essere validata rispetto a uno schema, che non intercetta un payload conforme ma trasforma la fiducia implicita in un vincolo dichiarato.

2.3 Sintesi: dove si trova il difetto

Lette una accanto all'altra, le categorie descritte in questo capitolo si distinguono meno per il tema che per il luogo in cui il difetto lascia una traccia. Per alcune il difetto è scritto negli artefatti di progetto: una configurazione insicura sta nel file che dichiara la risorsa, come nella IaC misconfiguration e in API8; un limite mancante sta nel codice, nel proxy o nel contratto (API4); un URL fornito dall'utente finisce in una chiamata di rete (API7); la risposta di un servizio esterno viene usata senza essere validata (API10). Per queste categorie le fonti indicano già contromisure verificabili, dagli strumenti automatici per le configurazioni all'analisi statica del costo di una query [12].

Per le categorie di autenticazione e di autorizzazione (API1, API2, API3 e API5) gli artefatti mostrano solo una parte del problema. Si vede se un controllo manca, per esempio un oggetto recuperato senza legarlo al chiamante o una funzione riservata priva di verifica del ruolo, ma non se un controllo presente decida correttamente. La stessa tassonomia CWE distingue il controllo assente da quello che esiste ma decide male [20], e solo il primo lascia nel codice una traccia certa.

Per due categorie, infine, il difetto non è negli artefatti. In API6 lo stesso endpoint è sicuro in un contesto e vulnerabile in un altro, e a deciderlo è il modello di business che gli sta dietro [23]; in API9 il difetto è l'assenza di un inventario, che non è un comportamento del software [27].

Le fonti esaminate descrivono ciascuna categoria con precisione, ma una alla volta. Nessuna stabilisce, per l'intera lista, quali categorie si possano accertare leggendo gli artefatti, quali richiedano di osservare il sistema in esecuzione e quali dipendano

da conoscenze che gli artefatti non contengono. È da questa domanda che parte il Capitolo 3, che trasforma la distinzione in un criterio di classificazione e ne verifica le conseguenze costruendo uno strumento di rilevazione.

3.1 Approccio metodologico

3.1.1 Obiettivi e domande di ricerca

La rassegna del capitolo precedente restituisce un quadro disomogeneo: alcune categorie dell'OWASP API Security Top 10 dispongono di tecniche di rilevazione consolidate, altre sono trattate quasi solo in chiave dinamica, altre ancora non hanno una formulazione operativa che ne consenta il riconoscimento automatico. Il divario dipende da come si manifesta la violazione che definisce ciascuna categoria. Vi sono violazioni che lasciano una traccia strutturale negli artefatti di progetto (quali codice applicativo, definizioni infrastrutturali e specifica dell'interfaccia) e violazioni il cui riconoscimento presuppone di sapere quale uso dell'applicazione sia legittimo, informazione che nessuno di essi contiene.

Questa differenza corrisponde alla distinzione fra *vulnerabilità tecniche* e *vulnerabilità di logica di business* (*logic vulnerabilities*) [30, 31]. Le prime violano proprietà strutturali verificabili sugli artefatti di progetto; le seconde violano l'intento per cui il flusso applicativo è stato concepito. La stessa OWASP, nel motivare la categoria API6:2023, riconduce a questo carattere la resistenza dei difetti di logica di business

al rilevamento automatico: la correttezza è definita rispetto all'intento del dominio applicativo, non rispetto a una proprietà del codice [23].

Il presente lavoro applica sistematicamente tale distinzione alle dieci categorie dell'OWASP API Security Top 10 (2023) e ne verifica le conseguenze implementative con un'analisi multi-layer dei tre livelli indicati, integrata da evidenza dinamica dove la sola ispezione statica non basta. Su questa base si formula la domanda di ricerca principale.

Q RQ₁: *Quali categorie dell'OWASP API Security Top 10 (2023) sono riconducibili a vulnerabilità tecniche rilevabili mediante analisi statica multi-layer su codice applicativo, definizioni infrastrutturali e specifica dell'interfaccia, quali richiedono in aggiunta evidenza esterna agli artefatti, e di quale natura sia tale evidenza?*

La domanda si scompone in due interrogativi subordinati.

Q RQ_{1.1}: *Per le categorie riconducibili a vulnerabilità tecniche, quale combinazione di livelli di analisi è necessaria alla loro copertura, e quali condizioni di rischio emergono soltanto dalla congiunzione di segnali che, presi isolatamente, non costituiscono una violazione?*

Q RQ_{1.2}: *Per le categorie la cui proprietà violata non è interamente esprimibile sugli artefatti, quale componente resta riconoscibile staticamente e di quale natura è il residuo: accertabile sollecitando il sistema in esecuzione, colmabile estendendo il perimetro degli artefatti esaminati, esplicitabile come parametro dichiarato dell'analisi, oppure irriducibile alla conoscenza del dominio applicativo?*

3.1.2 Impostazione dello studio

Le domande non si risolvono sulla carta. Un criterio applicato con coerenza può dire quali categorie *sembrano* riconducibili a violazioni tecniche; se la riconduzione regga davvero lo si scopre solo costruendo i moduli di rilevazione e osservandoli all'opera su codice reale. Lo studio assume perciò una forma costruttiva.

Questo chiarisce il ruolo del sistema descritto nel seguito: è il banco di prova della classificazione, non l'oggetto della ricerca. Ogni categoria ricondotta a una violazione strutturale deve avere una regola che la riconosca; ogni categoria esclusa deve restare, coerentemente, senza modulo. Vale come evidenza soltanto ciò che è stato verificato: la classificazione secondo il criterio dichiarato, l'esito dei moduli su casi di prova e su un bersaglio realistico, e la motivazione documentata delle categorie non coperte. Una regola presente nel codice ma mai esercitata su un caso concreto non conta come copertura. Il capitolo formula il criterio, ne deriva i requisiti per lo strumento e si chiude con il disegno della valutazione; l'applicazione del criterio alle dieci categorie è presentata insieme ai risultati.

3.1.3 Il criterio di classificazione

Il criterio rende operativa la distinzione della Sottosezione 3.1.1. Per ogni categoria si individua la proprietà di sicurezza la cui violazione la definisce, attenendosi alla formulazione ufficiale OWASP [4], alle debolezze CWE associate e alla caratterizzazione emersa dalla rassegna.

Una categoria è detta *tecnica* quando esiste un predicato sugli artefatti di progetto che sia al tempo stesso necessario e sufficiente a stabilire la violazione, una volta fissati i parametri dichiarati dell'analisi. È detta *ibrida* quando esiste un predicato necessario ma non sufficiente: l'analisi statica riconosce una condizione senza la quale la violazione non può sussistere, ma la cui presenza non basta a stabilirla. È detta infine *di logica di business* quando non esiste alcun predicato necessario esprimibile sugli artefatti, poiché la proprietà violata è definita per riferimento all'intento applicativo.

È necessario delimitare quali parametri sia lecito fornire all'analisi: si ammettono unicamente configurazioni generali valide per l'intero sistema (come l'uso di canali cifrati o limiti di traffico attesi). Si escludono invece regole puntuali sui permessi degli utenti rispetto alle singole risorse, per evitare di inserire a priori la logica di autorizzazione che l'analisi stessa dovrebbe verificare.

Il residuo delle categorie ibride non è omogeneo, e distinguerne la natura è l'oggetto di RQ_{1.2}. Un residuo è *dinamico* quando la proprietà mancante si manifesta nella risposta del sistema a una sollecitazione mirata, tipicamente l'invio della stessa

richiesta con identità distinte. È *di perimetro* quando è colmabile estendendo l'analisi a un livello non ancora esaminato: il limite riguarda l'applicazione dell'analisi statica a un solo livello, non l'approccio in sé. È *di dichiarazione* quando richiede che qualcuno espliciti la configurazione attesa o il contenuto di un accordo esterno. È *irriducibile* quando nessuna osservazione del sistema, statica o dinamica, può restituirlo.

Il criterio presuppone infine che il sistema da valutare sia noto. La condizione non vale per API9:2023, che descrive l'esistenza di sistemi che l'organizzazione ignora di possedere: la categoria resta perciò fuori dalla tripartizione.

3.1.4 Requisiti derivati per lo strumento di rilevazione

La classificazione non descrive uno strumento, ma vincola ciò che uno strumento adeguato deve saper fare. I requisiti che seguono sono formulati come capacità richieste, indipendentemente dalle tecnologie; la loro traduzione in un'architettura concreta è mostrata nella Sezione 3.2 per la IaC misconfiguration e nelle sezioni seguenti per le altre categorie.

Il primo requisito è l'analisi su più livelli. Le proprietà individuate dal criterio non risiedono tutte nello stesso luogo: alcune si manifestano nel codice applicativo, altre nelle definizioni infrastrutturali, altre nella specifica dell'interfaccia, e nessun livello è da solo sufficiente. Uno strumento che esamini un solo livello non può quindi coprire il perimetro, e ciascun livello richiede una rappresentazione e una tecnica di ispezione proprie.

Il secondo è la correlazione fra i livelli. Alcune condizioni non sono osservabili in alcun singolo livello, ma nella discrepanza fra due di essi: un percorso presente nel codice e assente dal contratto non è una violazione in nessuno dei due luoghi, eppure la divergenza segnala una superficie esposta e non governata; l'assenza di un limite nel codice cambia significato a seconda che le definizioni infrastrutturali ne prevedano uno altrove. Lo strumento deve quindi mettere in relazione segnali di livelli diversi e derivarne giudizi che nessun livello esprime da solo. Questa capacità risponde a RQ_{1.1}.

Il terzo è la verifica dinamica: per le categorie che richiedono un riscontro a runtime, lo strumento deve poter inviare richieste reali all'API e analizzarne le

risposte. L'analisi del codice rileva infatti solo un rischio potenziale, che rimarrebbe incerto senza una conferma attiva.

Il quarto è la configurabilità dei parametri: i valori e le soglie utilizzati dall'analisi devono essere espliciti e modificabili dall'utente, per evitare che impostazioni predefinite e cablate nelle regole impongano scelte arbitrarie come se fossero vincoli universali.

Il quinto è la tracciabilità: ogni segnalazione deve essere riconducibile alla categoria di cui costituisce evidenza e al livello di analisi che l'ha generata, perché una segnalazione priva di tale riconduzione può indicare un difetto reale ma non contribuisce a rispondere alle domande di ricerca.

3.1.5 Disegno della valutazione

La valutazione ha due scopi distinti: controllare che ogni modulo riconosca la condizione per cui è stato scritto, e osservare il comportamento dello strumento nel suo insieme su codice che non è stato scritto per essere analizzato.

Al primo scopo servono coppie di casi di prova, ciascuna formata da una versione che contiene la condizione da rilevare e da una che non la contiene: il modulo deve segnalare la prima e non la seconda. Le coppie coprono tutti e tre i livelli di analisi, così che nessun livello sia esercitato solo indirettamente attraverso il bersaglio realistico. Poiché casi di prova e regole hanno lo stesso autore, questa prova stabilisce che l'implementazione fa ciò che si intendeva, non che funzioni su codice reale.

Al secondo scopo serve OWASP crAPI [32], applicazione volutamente vulnerabile mantenuta dalla OWASP Foundation e quindi fissata a una revisione precisa per rendere i risultati riproducibili. Non essendo scritta dall'autore, riduce la circolarità della prima prova; resta però un bersaglio didattico, costruito perché le sue vulnerabilità siano riconoscibili, e i risultati non si possono estendere senza cautela ad applicazioni reali.

Per i casi di prova il riferimento di verità è noto per costruzione; per crAPI è stato ricostruito a mano, individuando nel codice i punti in cui si manifestano le vulnerabilità documentate dal progetto, ed è riportato per esteso perché i risultati vadano letti rispetto a ciò che si è assunto come vero. Una segnalazione conta co-

me corrispondente a una voce del riferimento quando riguarda la stessa categoria OWASP e lo stesso punto di esposizione dell'interfaccia, identificato da metodo e percorso normalizzato, qualunque sia il livello di analisi che l'ha prodotta; questa convenzione è stata fissata prima di osservare i risultati. Su questa base si registra, per ogni categoria, se i moduli segnalano i casi che contengono la condizione e non quelli che ne sono privi, e si distinguono le categorie senza segnalazioni in cui ciò discende dalla classificazione da quelle in cui indica una copertura mancante.

3.2 Rilevamento delle IaC misconfiguration

Questa sezione applica il metodo alla prima delle classi in esame, la *IaC misconfiguration* (Sottosezione 2.1.1): le configurazioni difettose dichiarate in file ispezionabili prima del rilascio. Qui tali file sono la definizione dell'infrastruttura come codice (i manifest Terraform, affiancati da Dockerfile e manifest Kubernetes gestiti dal medesimo strumento) e l'analisi è interamente statica: nessuna risorsa viene creata né interrogata.

Dei quattro scenari con cui OWASP illustra la categoria, due ricadono nel perimetro del sistema e due ne restano fuori. Rientrano il secondo e il quarto, perché in entrambi la configurazione difettosa è dichiarata in un artefatto ispezionabile — la direttiva che abilita il directory listing nella configurazione del server, i permessi di condivisione fissati nel manifest della risorsa di storage — e può quindi essere individuata prima del rilascio. Restano fuori il primo e il terzo: le applicazioni di esempio dimenticate in produzione non compaiono in alcun artefatto del progetto e presuppongono un inventario degli host effettivamente in esecuzione, mentre i messaggi di errore troppo dettagliati emergono soltanto dalle risposte prodotte a runtime.

L'esposizione segue il percorso di una segnalazione: l'architettura entro cui lo scanner opera, lo strumento adottato e le policy pertinenti, l'adapter che ne traduce l'uscita nel modello di dominio e, infine, l'aggregazione con il calcolo del punteggio di rischio. Gli esiti della validazione sono riportati nel Capitolo 4.

3.2.1 Architettura del sistema

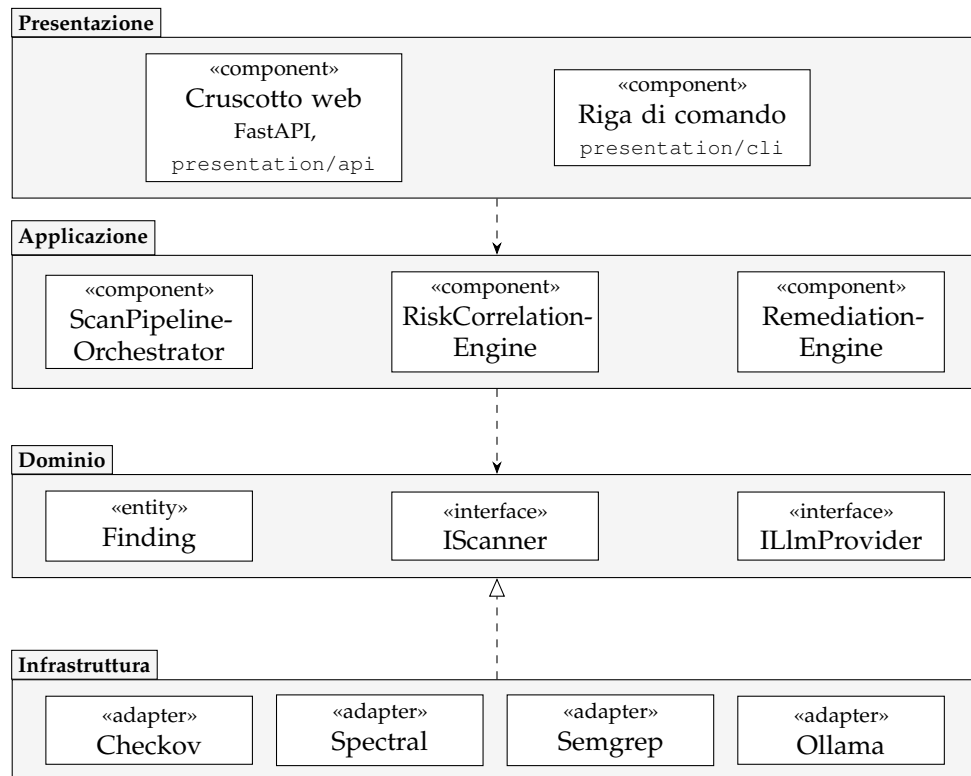


Figura 3.1: Diagramma UML dei componenti del sistema, raggruppati per livello architetturale. Le frecce tratteggiate indicano dipendenze; quella con la punta triangolare vuota indica la realizzazione delle interfacce di dominio da parte degli adapter.

Il sistema è organizzato secondo i principi della *Clean Architecture*: un livello di dominio che definisce le entità e i contratti, un livello applicativo che orchestra l’analisi, un livello infrastrutturale che incapsula gli strumenti esterni e un livello di presentazione che espone i risultati. La Figura 3.1 mostra i componenti raggruppati per livello e le dipendenze fra i livelli: la presentazione dipende dall’applicazione, l’applicazione dal dominio, e l’infrastruttura realizza le interfacce che il dominio dichiara. Il livello di presentazione offre due vie d’accesso alla stessa pipeline: un’interfaccia a riga di comando, che registra gli adapter, esegue l’orchestratore e scrive i rapporti su file, e un cruscotto web basato su FastAPI, che espone la stessa esecuzione come servizio e presenta i risultati in pagine interattive.

La separazione risponde direttamente alla tracciabilità richiesta nella Sottosezione 3.1.4: qualunque sia lo strumento che l’ha prodotta, ogni anomalia viene ricondotta a un’unica entità di dominio, il *Finding*, che la Figura 3.2 mostra insieme ai suoi og-

getti valore, alle enumerazioni di severità e natura, all'interfaccia `IScanner` con l'adapter che la realizza e al motore di correlazione.

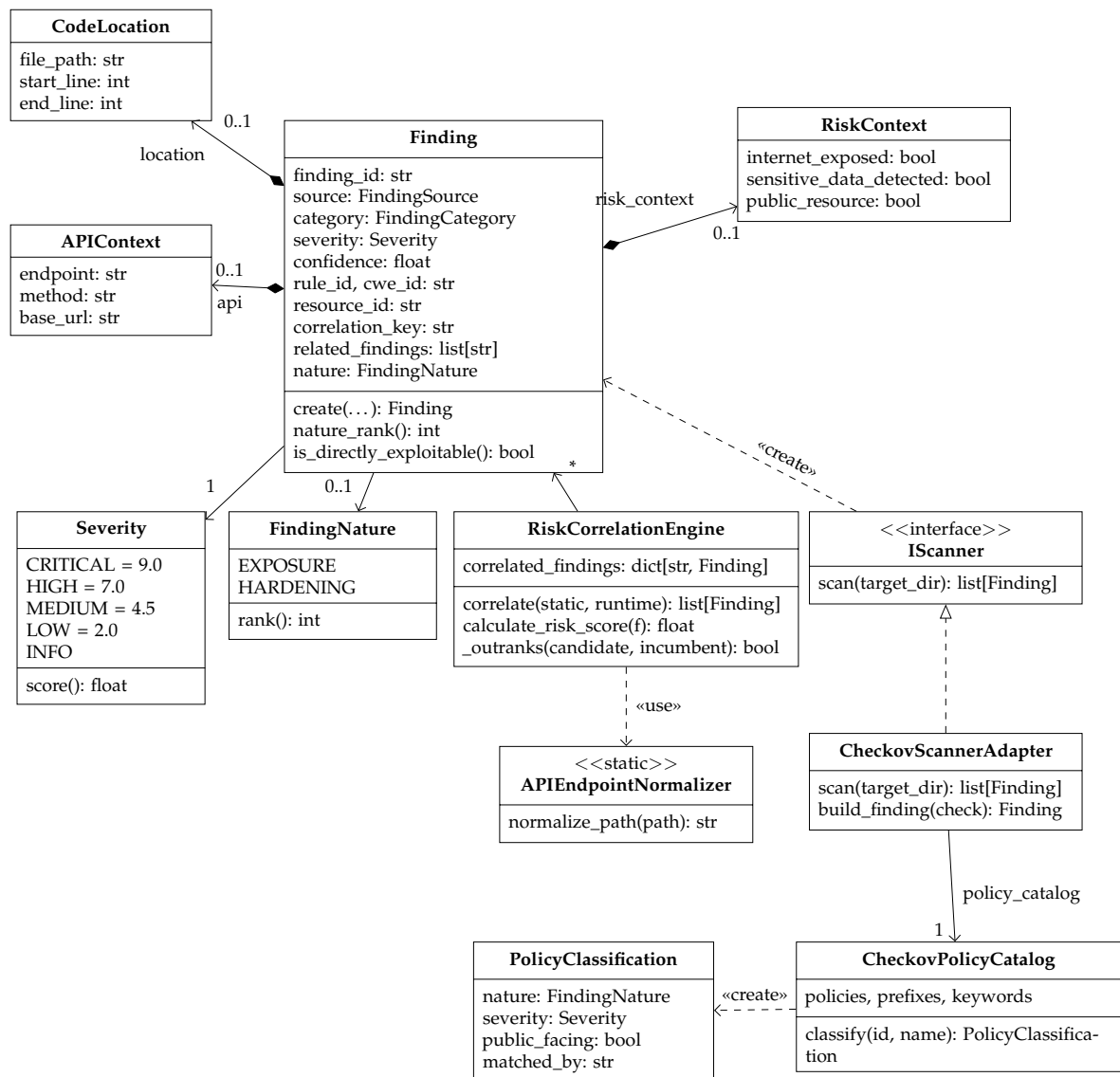


Figura 3.2: Diagramma UML delle classi del dominio e del motore di correlazione. `Severity` e `FindingNature` sono enumerazioni; `APIEndpointNormalizer` espone solo metodi di classe. Gli adapter di Semgrep e Spectral, non trattati, realizzano la stessa interfaccia.

Un `Finding` risponde a tre domande. Da dove viene: la sorgente, la regola che l'ha generato e i dati grezzi dello strumento. Dove si trova: la posizione nel file, la risorsa infrastrutturale interessata e una chiave di correlazione. Quanto pesa: la categoria, la severità su cinque gradi (CRITICAL, HIGH, MEDIUM, LOW, più INFO per le note informative) e un grado di confidenza fra 0 e 1. A queste si aggiunge la *natura*

del rischio, che distingue un'*esposizione*, quando la configurazione apre un varco sfruttabile direttamente, da un *irrobustimento*, quando manca una linea di difesa in profondità che da sola non costituisce un accesso. Quando la regola non permette di stabilire a priori da quale parte stia la segnalazione, la natura resta indefinita (un valore nullo nel dominio) e la segnalazione non classificata.

L'interfaccia astratta `IScanner` formalizza il contratto a cui ogni componente di analisi deve conformarsi: un metodo `scan` che riceve il percorso del bersaglio e restituisce una lista di `Finding`. È implementata dagli *adapter* che avvolgono strumenti esterni e hanno il compito di eseguire lo strumento, leggerne l'uscita e tradurla nel modello di dominio. L'adapter di cui si occupa questa sezione è quello di Checkov [33], che analizza le definizioni infrastrutturali; la stessa interfaccia ha altre due realizzazioni, Semgrep per il codice applicativo e Spectral per la specifica dell'interfaccia, che servono altre categorie e restano fuori da questa sezione. L'orchestratore conosce gli scanner solo attraverso questa interfaccia, e l'esecuzione procede in due fasi: gli scanner registrati vengono eseguiti sul repository bersaglio, poi le segnalazioni raccolte vengono consegnate al motore di correlazione, che le aggrega per chiave e assegna a ciascuna un punteggio di rischio. Aggiungere un controllo significa registrare un nuovo adapter, senza modificare l'orchestratore.

3.2.2 Lo scanner Checkov e la traduzione nel modello di dominio

Checkov [33] è un analizzatore statico per Infrastructure-as-Code che legge i file di definizione (tra cui Terraform [34], CloudFormation, Kubernetes, Dockerfile, Helm e specifiche OpenAPI), ne costruisce un grafo di risorse e vi valuta un catalogo di policy. Le policy sono di due tipi: controlli sugli attributi di una singola risorsa, scritti in Python e identificati da `CKV_provider_n`, e controlli sulle relazioni fra risorse, espressi in YAML sul grafo e identificati da `CKV2_provider_n`; a questi si aggiunge un riconoscitore di segreti che esamina qualunque file e produce identificativi `CKV_SECRET_n`. Ogni valutazione di una policy su una risorsa dà un esito superato, fallito o non determinabile, con il file e l'intervallo di righe della risorsa.

Per lo scenario in perimetro, riguardante i permessi di condivisione aperti su risorse di storage, il catalogo contiene controlli che coprono le tre vie attraverso cui un

bucket S3 può risultare pubblico: la lista di controllo d’accesso, la policy del bucket e il blocco dell’accesso pubblico a livello di account o di bucket. La Tabella 3.1 li elenca insieme ai controlli che Checkov valuta sulle stesse risorse senza che riguardino l’esposizione (registrazione degli accessi, versioning, cifratura con chiave gestita, replica, ciclo di vita, notifiche) e che lo strumento colloca sullo stesso piano dei primi. La distinzione fra esposizione e irrobustimento, essenziale per questo lavoro, nel catalogo di Checkov non esiste: è il sistema a introdurla attraverso un catalogo proprio delle policy, descritto nel seguito.

Policy	Risorsa Terraform	Condizione verificata
<i>Esposizione dello storage</i>		
CKV_AWS_20	aws_s3_bucket	ACL che concede lettura pubblica
CKV_AWS_53–56	aws_s3_bucket_public_access_block	i quattro flag del blocco dell’accesso pubblico non attivi
CKV_AWS_70	aws_s3_bucket_policy	policy che concede un’azione a qualunque principal
CKV2_AWS_6	aws_s3_bucket	bucket privo di un blocco dell’accesso pubblico attivo (controllo sul grafo)
CKV2_AWS_65	aws_s3_bucket_ownership_controls	liste di controllo d’accesso non disabilitate
<i>Irrobustimento dello storage, senza esposizione</i>		
CKV_AWS_18	aws_s3_bucket	registrazione degli accessi assente
CKV_AWS_21	aws_s3_bucket	versioning non abilitato
CKV_AWS_145	aws_s3_bucket	cifratura non basata su chiave KMS
CKV_AWS_144	aws_s3_bucket	replica fra regioni assente
CKV2_AWS_61, 62	aws_s3_bucket	ciclo di vita e notifiche di evento assenti
<i>Altre risorse del bersaglio di prova</i>		
CKV_AWS_59	aws_api_gateway_method	metodo esposto senza autorizzazione
CKV_AWS_62, 63	aws_iam_role_policy	policy con privilegi amministrativi o azioni *
CKV_AWS_45	aws_lambda_function	segreto cablato nelle variabili d’ambiente
CKV_AWS_169	aws_sns_topic_policy	policy del topic aperta a qualunque principal

Tabella 3.1: Policy di Checkov 3.2.459 pertinenti allo scenario in perimetro e alle altre risorse del bersaglio di prova. Le denominazioni sono tradotte dai nomi ufficiali dei controlli.

L’adapter esegue Checkov sulla sola directory bersaglio richiedendo l’uscita in formato JSON, e traduce ciascun controllo fallito in un Finding (Figura 3.3). La categoria è derivata tramite parole chiave da identificativo, nome ufficiale e tipo di risorsa. Natura, severità ed esposizione pubblica provengono invece dal *catalogo delle*

policy: un file YAML esterno che costituisce parametro dell'analisi e non costante del codice, in accordo con la configurabilità richiesta nella Sottosezione 3.1.4. Il catalogo risolve ciascun controllo secondo la priorità riassunta nella Tabella 3.2 (mappatura esplicita su 99 controlli, prefissi o parole chiave, con ripiego predefinito su segnalazione non classificata `MEDIUM`).

La confidenza del Finding non misura il controllo, che è deterministico sul file, ma la precisione con cui il catalogo lo ha interpretato: 1 per la mappatura esplicita, 0,9 per il prefisso, 0,8 per le parole chiave, 0,6 per il ripiego predefinito. Il Finding include titolo, intervallo di righe e record grezzo. Se il controllo è contrassegnato come pubblico, l'adapter vi associa un contesto di risorsa esposta a Internet e pubblica; se riguarda segreti o cifratura a riposo, un contesto di dati sensibili. Come chiave di correlazione adotta l'ultimo segmento del nome della risorsa (ad esempio `bucket_1_public_acl` per `aws_s3_bucket.bucket_1_public_acl`), consentendo di raggruppare i rilievi riferiti alla medesima entità logica definita dallo sviluppatore.

Regola	Come risolve il controllo	Esempio
Mappatura esplicita	identificativo presente nel catalogo, con natura, severità ed eventuali flag di esposizione pubblica e di dati sensibili	CKV_AWS_20: esposizione, CRITICAL, pubblica
Prefisso	famiglia di controlli riconosciuta dall'inizio dell'identificativo	CKV_SECRET_*: esposizione, HIGH, dati sensibili
Parole chiave	termini cercati nel nome ufficiale del controllo; i gruppi difensivi precedono quelli di esposizione	<i>logging, versioning</i> : irrobustimento, LOW
Predefinito	nessuna regola soddisfatta	non classificato, MEDIUM

Tabella 3.2: Ordine con cui il catalogo delle policy assegna natura, severità e contesto a un controllo fallito di Checkov; la regola che risolve determina anche la confidenza. Le esposizioni hanno severità predefinita `HIGH`, gli irrobustimenti `LOW`, salvo indicazione esplicita.

3.2.3 Correlazione e punteggio di rischio

Il motore di correlazione indicizza le segnalazioni per chiave (nel caso delle segnalazioni infrastrutturali, il nome della risorsa) e fonde quelle che la condividono in un'unica voce. La rappresentante è scelta per natura e poi per severità: un'esposi-

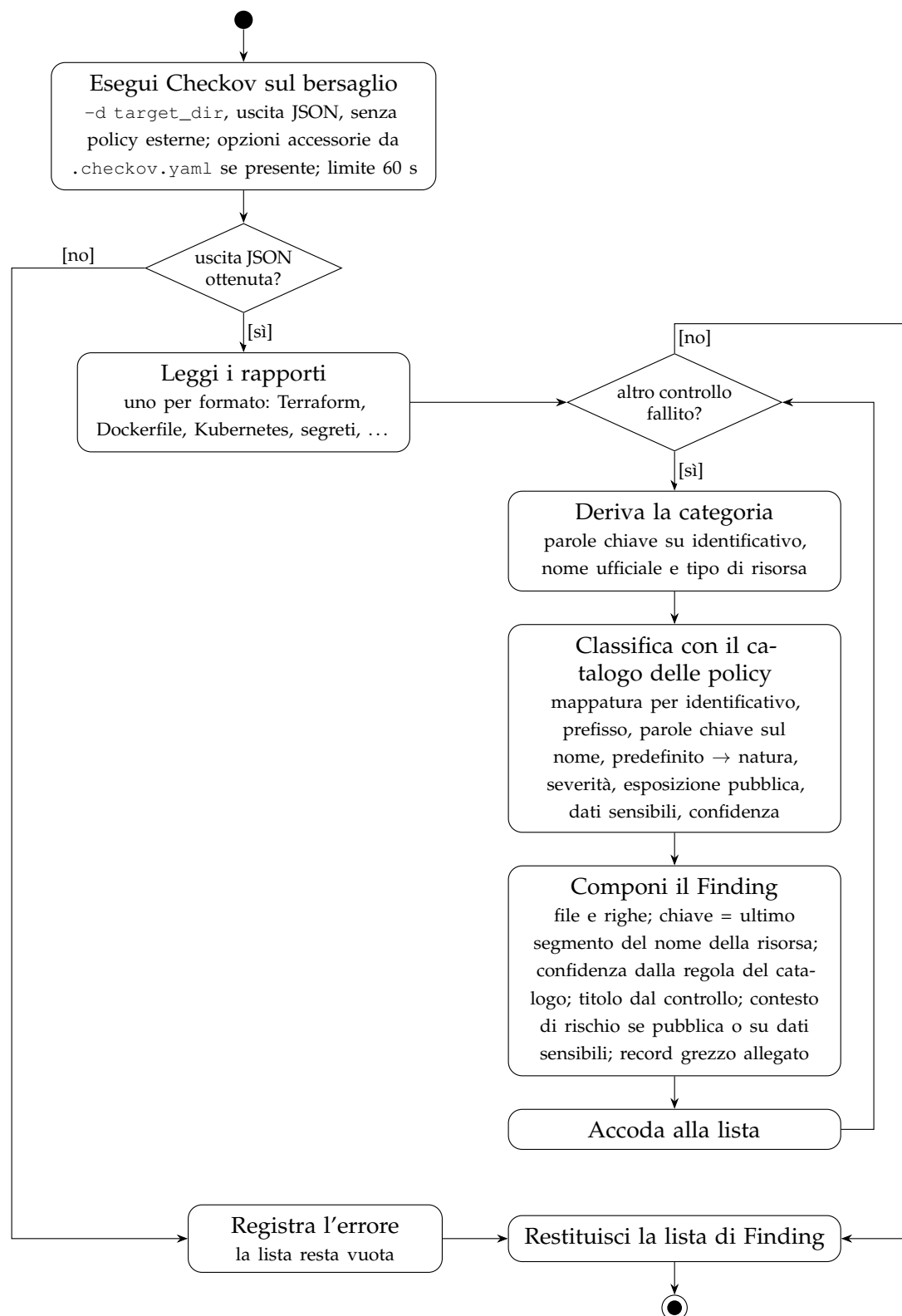


Figura 3.3: Diagramma UML di attività del metodo `scan` dell’adapter di Checkov. Ogni controllo fallito diventa un Finding: la categoria è derivata da identificativo, nome e tipo di risorsa; natura, severità ed esposizione pubblica provengono dal catalogo delle policy.

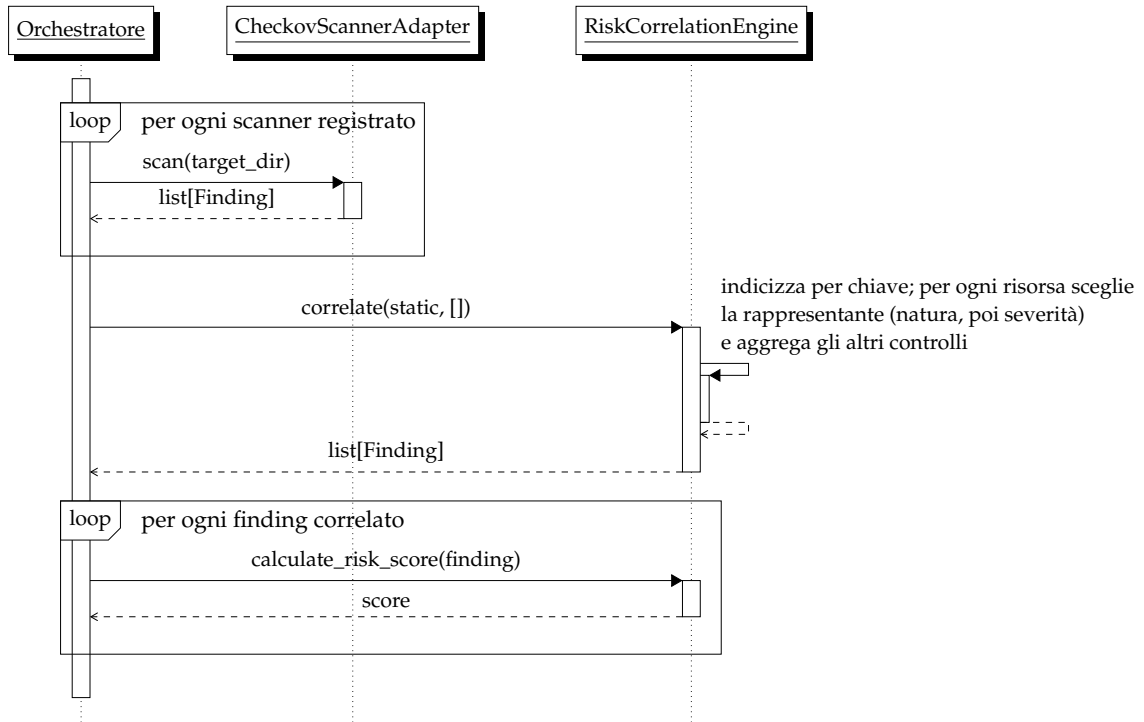


Figura 3.4: Diagramma UML di sequenza della pipeline statica dell’orchestratore. Il secondo argomento di `correlate`, riservato alle segnalazioni a runtime di altri moduli, è vuoto.

zione precede una segnalazione non classificata, che precede un irrobustimento; a parità di natura prevale la severità maggiore. Un controllo di natura inferiore non altera mai la severità della rappresentante e resta come sotto-elemento della voce, con la propria natura e severità: nulla viene scartato, e una sola esecuzione restituisce l’intero insieme delle condizioni da correggere. La Figura 3.4 riporta la sequenza della pipeline statica.

Il punteggio di rischio è un modello pesato proprio. Il CVSS sarebbe stato la scelta naturale, ma richiede un vettore che nessuno degli strumenti integrati produce:

$$R = \min(10, 0,6 S + 0,2 \cdot 10 C + 0,2 X), \quad (3.2.1)$$

Tre termini concorrono al risultato. S è il valore numerico della severità: 10 per CRITICAL, 7,0 per HIGH, 4,5 per MEDIUM, 2,0 per LOW, 0 per INFO. $C \in [0,1]$ è la confidenza dichiarata dalla sorgente: per Checkov la precisione della regola del catalogo, per ZAP il livello riportato dall’alert, per Semgrep 0,95 se il presidio di autenticazione è stato trovato e 0,7 se la sua assenza è dedotta; sale a 1 quando un riscontro a runtime conferma la segnalazione. X è il contesto, somma dei contributi

dichiarati dalla sorgente: 4 per una risorsa raggiungibile da Internet, 2 se è pubblica, 4 se coinvolge dati sensibili, 3 quando nessun contesto è dichiarato. Per gli irrobustimenti `X` vale sempre 0: una linea di difesa mancante non è un vettore d'accesso. Con `CRITICAL` pari a 10 il massimo teorico coincide con il limite superiore, che interviene solo quando concorrono tutti i contributi di contesto.

In pratica, per le segnalazioni infrastrutturali (confidenza 1): un'esposizione `CRITICAL` su risorsa pubblica vale 9,2, una `CRITICAL` su dati sensibili 8,8, una `HIGH` senza contesto 6,8, un irrobustimento `LOW` 3,2. Pesi, punteggi di severità, contributi di contesto e mappe di confidenza risiedono in un file `YAML` di configurazione; i valori riportati sono i predefiniti nel codice.

Il cruscotto ordina le voci per natura e poi per punteggio decrescente, mostrato accanto alla severità, e sotto ciascuna elenca i controlli aggregati. Ogni voce e ogni controllo ricevono un'indicazione correttiva: da una base di conoscenza locale indicizzata per controllo oppure, in sua assenza, da un consiglio generato dall'intelligenza artificiale con un modello linguistico eseguito localmente tramite Ollama. Il motore di remediation dipende soltanto dall'interfaccia di dominio `ILlmProvider`; l'adapter di Ollama la realizza e viene iniettato dal punto di composizione, sicché il livello applicativo non conosce l'infrastruttura, come per gli scanner.

3.3 Categorie ibride: analisi statica e residuo dinamico

Le categorie raccolte in questa sezione sono quattro: la Broken Object Level Authorization (API1), la Broken Authentication (API2), la Broken Object Property Level Authorization (API3) e la Broken Function Level Authorization (API5). Tre riguardano l'autorizzazione a granularità diverse, cioè l'oggetto a cui si accede, le sue singole proprietà e la funzione che si invoca; la quarta riguarda il passo che le precede, il riconoscimento di chi effettua la richiesta. Secondo il criterio della Sottosezione 3.1.3 sono tutte categorie ibride, ed è su di esse che il sistema cerca una risposta concreta a `RQ1.2`.

Per ciascuna esiste infatti un predicato sugli artefatti necessario ma non sufficiente. Un oggetto recuperato senza legarlo al chiamante, un token decodificato senza verificarne la firma, un oggetto restituito con tutti i suoi campi riservati, una funzione

amministrativa priva di controllo di ruolo sono condizioni senza le quali la violazione non può esistere, e l'analisi statica le trova. Quando il controllo è presente, invece, gli artefatti non dicono se funzioni: il filtro sul proprietario può usare il campo sbagliato, un token scaduto può essere accettato, un controllo di ruolo può non bloccare nulla. Proprietà di questo tipo si osservano solo nella risposta del sistema a una sollecitazione mirata, di solito la stessa richiesta inviata con identità diverse, e formano il residuo che $RQ_{1,2}$ chiama *dinamico*. È questa forma comune del residuo, più della vicinanza fra i temi, a giustificare una trattazione unica.

Tutti i moduli distinguono tre tipi di controllo, che nel codice si somigliano perché spesso sono decoratori o middleware applicati alla stessa rotta: l'autenticazione stabilisce chi è il chiamante, il controllo di proprietà se l'oggetto richiesto gli appartiene, il controllo di ruolo che cosa gli è permesso fare. Ogni modulo cerca quello pertinente alla propria categoria (il legame fra oggetto e chiamante per la BOLA, il ruolo attorno a una proprietà per la BOPLA, il ruolo sulla funzione per la BFLA), mentre il modulo della Broken Authentication esamina il meccanismo stesso che riconosce il chiamante. Per le categorie di autorizzazione la sola autenticazione non vale mai come protezione, perché qualunque utente registrato la supera.

Il criterio vieta inoltre di dire all'analisi quali oggetti, proprietà o funzioni siano riservati, perché significherebbe consegnarle in anticipo la logica di autorizzazione che deve verificare. La configurazione contiene solo elementi generali, come l'indirizzo del bersaglio, le identità di prova con i loro ruoli o i nomi dei campi che esprimono un privilegio, e ciò che è protetto va dedotto dagli artefatti.

I moduli si separano invece nel trattamento del residuo. Nella BOLA e nella BOPLA la segnalazione statica è un'ipotesi: un livello dinamico invia richieste reali con identità di ruolo diverso e il suo esito torna sulla segnalazione, confermandola o smentendola. Nella Broken Authentication il rapporto si rovescia, perché la parte statica serve soprattutto a ricostruire il modello di autenticazione che orienta le prove dinamiche, e una sola regola statica copre il caso in cui il sistema non si lascia interrogare. Il modulo della BFLA, infine, resta statico: riconosce l'assenza del controllo di ruolo ma non invoca le funzioni per accertarla, e il residuo dinamico rimane aperto.

Struttura comune dei rilevatori I quattro moduli non condividono un'unica architettura. Seguono due impianti, uno per l'analisi statica a livelli e uno per le pipeline a fasi, e il modulo della BOLA li usa entrambi, perché affianca a un rilevatore statico un livello dinamico. Conviene descriverli qui una volta sola, insieme al modo in cui i risultati raggiungono il resto del sistema, e lasciare alle sottosezioni ciò che è proprio di ciascuna categoria.

Il rilevatore statico della BOLA e quello della BFLA sono costruiti sullo stesso schema: un punto d'ingresso, un componente per ciascun livello di analisi, un componente per ciascuna regola e un modello dei dati comune a livelli e regole. Il punto d'ingresso esegue i livelli in sequenza, il codice sorgente sempre e il contratto OpenAPI solo quando è disponibile; la BFLA vi aggiunge un livello dedicato ai file di configurazione. Il codice Python e JavaScript/TypeScript è letto attraverso l'albero sintattico prodotto da tree-sitter [35] e non con ricerche testuali, così che commenti, formattazione e decorator sovrapposti non alterino il risultato. Il primo passo associa ogni funzione che gestisce una rotta, detta *handler*, al percorso e ai metodi che espone, tenendo conto dei prefissi dei raggruppamenti di rotte (i *Blueprint* di Flask) e, in Express, dei middleware passati alla registrazione della rotta, fra i quali vanno cercati i controlli. Su questa mappa operano le regole, e ogni segnalazione ha la stessa struttura nei due moduli: regola e debolezza CWE, severità, file e riga, endpoint e metodi, protezione trovata e protezione mancante, istruzione che fa da evidenza, confidenza e livello che l'ha prodotta.

Il livello del contratto pone all'interfaccia dichiarata le stesse domande poste al codice, ma conta soprattutto come termine di confronto. Un'operazione che il contratto dichiara protetta e che il codice non protegge smentisce una garanzia promessa, mentre un endpoint presente nel codice e assente dal contratto indica una superficie che nessuno ha documentato. È la correlazione fra livelli richiesta da RQ_{1.1}, applicata a categorie che non si esauriscono negli artefatti, e la stessa idea ritorna nelle regole contrattuali della BOPLA. Poiché più regole e più livelli possono colpire lo stesso endpoint, le segnalazioni vengono poi deduplicate: fra quelle che cadono sullo stesso punto sopravvive la segnalazione della regola con priorità più alta, dove la priorità misura la precisione della diagnosi e non la gravità, mentre le regole che misurano un aspetto diverso da quello conteso restano fuori dalla competizione e

si affiancano alla segnalazione superstite. Il risultato è un rapporto che riassume le segnalazioni per severità, regola e livello e che, su richiesta, annota nel contratto OpenAPI le operazioni coinvolte.

La Broken Authentication e la BOPLA seguono invece un impianto a fasi, che parte dal repository e arriva al sistema in esecuzione. Una fase di scoperta raccoglie ciò che gli artefatti, ed eventualmente il traffico osservato, dicono del bersaglio, una fase di sintesi ne ricava un modello (il modello di autenticazione nel primo caso, l'insieme delle proprietà protette nel secondo), una batteria di prove dinamiche lo mette alla prova e un rapporto finale, in formato JSON e Markdown, ne raccoglie gli esiti, riassunti anche in un punteggio di robustezza che parte da cento e scende per ogni debolezza accertata. I due moduli condividono anche una parte dell'implementazione: la BOPLA riusa la configurazione del bersaglio e l'analizzatore del codice della Broken Authentication, costruito su tree-sitter e in grado di coprire in modo uniforme otto linguaggi. Anche il livello dinamico della BOLA procede per fasi, dalla scoperta degli endpoint alla preparazione dei dati fino all'attacco.

Il formato delle segnalazioni, infine, non è lo stesso per tutti i moduli. Il Finding di dominio della Sottosezione 3.2.1 è prodotto direttamente soltanto dal livello dinamico della BOLA, i cui esiti raggiungono il motore di correlazione della Sottosezione 3.2.3 attraverso la chiave formata dal metodo e dal percorso normalizzato dell'endpoint. Il rilevatore statico della BOLA e il modulo della BOPLA dispongono di una traduzione verso lo stesso modello che conserva questa chiave, per ora esercitata soltanto dalle prove automatiche, mentre la Broken Authentication e la BFLA producono esclusivamente un rapporto nel proprio formato. Nella panoramica del cruscotto i moduli vengono quindi eseguiti in sequenza e i loro risultati presentati modulo per modulo, senza passare per il motore di correlazione. La validazione dei moduli e i limiti emersi sono discussi nel Capitolo 4.

3.3.1 API1 – Il legame fra oggetto e chiamante

Una *Broken Object Level Authorization* [5] (Sottosezione 2.2.1) si verifica quando un endpoint riconosce chi lo invoca, ma non controlla che l'oggetto richiesto appartenga a quell'utente. Un esempio tipico è un endpoint come `GET /api/orders/{id}`:

è protetto dal login, ma a un utente basta sostituire nel percorso l'identificativo del proprio ordine con quello di un ordine altrui per leggerne il contenuto.

Il modulo implementato lavora su due livelli : Il livello statico cerca nel codice gli handler che accedono a un oggetto senza legarlo all'identità del chiamante. Quando questo legame manca, il difetto è facilmente verificabile. Quando invece c'è, il codice non basta a dire se sia corretto, infatti il campo usato potrebbe non esprimere davvero l'appartenenza, il confronto potrebbe essere scritto nel verso sbagliato, oppure il proprietario potrebbe essere un valore che il client è in grado di falsificare. Per questo il modulo comprende anche un livello dinamico, che invia richieste reali con identità diverse e ne osserva le risposte. I due livelli comunicano attraverso una chiave comune, formata dal metodo e dal percorso normalizzato dell'endpoint.

Il livello statico Il livello statico analizza applicazioni Flask, FastAPI ed Express. Per prima cosa stabilisce quali parametri di percorso fanno riferimento a un oggetto: sono quelli tipizzati come interi e quelli il cui nome richiama un identificativo, come `id`, `order_id`, `uuid` o `slug`. È la stessa euristica usata a runtime dalla fase di discovery, e permette di ignorare i parametri che non indicano un oggetto, come il termine di una ricerca.

Poi legge istruzione per istruzione il corpo di ogni handler, ripulito da commenti e docstring, e ne ricava i *fatti di ownership*. Questi fatti rispondono a tre domande: dove un identificativo del percorso viene usato per leggere o modificare un oggetto, anche passando per una variabile intermedia; dove viene letta l'identità del chiamante, per esempio `current_user` in Flask o `req.user` in Express; quali legami collegano l'oggetto a quell'identità. Un legame può essere una query che filtra anche sull'identità, un confronto fra il proprietario dell'oggetto e l'utente corrente, la chiamata a una funzione che verifica la proprietà oppure un decoratore dedicato.

Per questi controlli sono state definite 3 euristiche : il primo riguarda le condizioni composte, che vengono spezzate nelle singole clausole: così una condizione che nomina sia l'identità sia l'identificativo, senza però metterli in relazione, non viene scambiata per un controllo. Il secondo riguarda i percorsi annidati, dove contano anche i legami fra identificativi: un task cercato all'interno di un progetto già validato è protetto, un task cercato con il solo identificativo no. Il terzo riguarda le funzioni di

supporto definite nello stesso file. Spesso sono proprio loro a recuperare l'oggetto e a controllarne il proprietario, perciò il modulo ne riassume il comportamento e lo attribuisce all'handler che le chiama.

Le regole statiche Sui fatti di ownership operano le cinque regole: OA-001 e OA-002 coprono il caso di base, cioè un handler che accede all'oggetto con il solo identificativo inviato dal client: la prima si applica agli handler autenticati, la seconda a quelli privi di qualsiasi contesto di identità. OA-003 confronta fra loro gli handler della stessa risorsa. Se un metodo filtra sul proprietario e un altro no, è il codice stesso a mostrare che il controllo era un requisito noto, ed è per questo che la regola ha la confidenza più alta. OA-004 si occupa dei riferimenti annidati. OA-005, infine, segnala un identificativo numerico, e quindi prevedibile, esposto senza autenticazione.

Più regole possono scattare sullo stesso endpoint. In questo caso sopravvive la segnalazione della regola più precisa, secondo l'ordine $OA-003 > OA-004 > OA-001 > OA-002$. Una cancellazione non protetta, per esempio, soddisfa sia OA-003 sia OA-001: viene riportata come OA-003, e OA-001 resta come riferimento all'interno della segnalazione. OA-005 non partecipa a questa selezione, perché descrive l'identificativo e non il controllo, e si affianca alle altre segnalazioni. A differenza della BFLA, le segnalazioni sono raggruppate per file oltre che per chiave di correlazione: due servizi dello stesso repository possono esporre lo stesso percorso con implementazioni diverse, e in quel caso non si tratta di duplicati.

Il confronto con il contratto Quando è disponibile la specifica OpenAPI, ogni segnalazione viene confrontata con il requisito di sicurezza, locale o globale, dell'operazione corrispondente. I casi possibili sono tre. Nel primo, il più interessante, il contratto dichiara l'operazione autenticata ma il codice non lega l'oggetto al chiamante. La regola di Spectral [36] dedicata ad API1 controlla che le operazioni su un identificativo dichiarino un requisito di sicurezza. Il livello statico copre esattamente questo punto cieco, e la confidenza della segnalazione cresce di 0,10 perché il codice smentisce una garanzia del contratto. Nel secondo caso anche il contratto è privo di protezione, e la segnalazione si aggiunge a quella di Spectral sulla stessa chiave. Nel terzo l'handler non compare affatto nel contratto: si tratta di un possibile endpoint

non documentato, che il livello dinamico non attaccherebbe, perché costruisce il proprio inventario dalla specifica.

Il livello dinamico Il livello dinamico applica il *differential testing*: invia la stessa richiesta con identità diverse e ne confronta le risposte. Le identità sono tre: una *vittima* con ruolo ordinario, un *pari* con lo stesso ruolo e un'identità *privilegiata* con ruolo amministrativo. Un file di configurazione, che fa da contratto con l'applicazione in prova, indica come ottenerle, insieme all'indirizzo del bersaglio e agli endpoint di supporto. I token possono essere richiesti a un identity provider come Keycloak [37], forniti come valori statici o ricavati dal traffico; le credenziali, invece, vengono lette solo da variabili d'ambiente. Tutte le richieste passano per OWASP ZAP [38], usato come proxy, che per ogni esito vulnerabile avvia anche una scansione attiva dell'URL coinvolto.

L'elenco degli endpoint da provare arriva dalla pipeline oppure viene ricavato dalla specifica OpenAPI. I parametri di percorso sono normalizzati nel segnaposto `{id}` e si tengono solo gli endpoint che ne contengono almeno uno. Nei percorsi annidati, come `/projects/{id}/tasks/{id}`, il bersaglio è l'ultimo identificativo. L'eventuale traffico catturato aggiunge altri endpoint e altri riferimenti a oggetti, presenti nel percorso, nella query o nel corpo.

Per attaccare servono risorse che appartengano a ciascuna identità, e il modulo le prepara in due modalità. In *Lab Mode* il bersaglio collabora: espone alcuni endpoint di prova, con cui il modulo crea le risorse di ogni identità. Attorno a ogni metodo che modifica i dati (POST, PUT, PATCH, DELETE) il modulo salva lo stato e poi lo ripristina, così che un attacco riuscito, per esempio una cancellazione, non alteri le prove successive. In *Assessment Mode* il bersaglio non collabora. Identità, ruoli e risorse possedute si ricavano dal traffico osservato, e i token JWT vengono decodificati senza verificarne la firma, perché si tratta di traffico già avvenuto. I metodi che modificano i dati sono esclusi, salvo autorizzazione esplicita, perché in questa modalità lo stato non può essere ripristinato.

Per ogni endpoint e metodo il modulo prova tre scenari. Nella BOLA *orizzontale* il pari tenta di accedere a una risorsa della vittima. Nella BOLA *verticale* la vittima tenta di accedere a una risorsa dell'identità privilegiata. Il terzo scenario è un accesso

di controllo: l'amministratore accede alla risorsa della vittima, cosa che il sistema dovrebbe consentire. In ciascuno scenario partono tre richieste: quella del proprietario, che fornisce la risposta di riferimento; quella dell'attaccante, che usa il proprio token sulla risorsa altrui; una richiesta anonima, senza credenziali. Gli identificativi vengono sostituiti ovunque compaiano, cioè nel percorso, nella query e nel corpo.

Il verdetto La risposta ricevuta dall'attaccante viene valutata in due passi. Il primo stabilisce se la richiesta è stata bloccata. Lo è quando riceve 401, 403 o un'altra risposta di errore. Molte applicazioni, però, rifiutano l'accesso senza usare il codice di stato appropriato, per cui conta come bloccata anche una risposta di successo che contiene un messaggio di rifiuto, come *access denied* o *non autorizzato*: in questo caso si parla di *blocco mascherato*. Il secondo passo, se la richiesta non è stata bloccata, stabilisce se l'attaccante ha davvero ottenuto la risorsa. Si ha un *successo tecnico* quando il corpo della risposta coincide con quello ricevuto dal proprietario, al netto dei campi volatili che cambiano a ogni chiamata, come le date di aggiornamento, oppure quando contiene l'identificativo della risorsa della vittima. Il confronto avviene sulla struttura JSON e non sulla lunghezza della risposta, e l'elenco dei campi volatili è configurabile.

Un successo tecnico, però, non è ancora una vulnerabilità. L'amministratore che legge l'ordine di un utente ottiene la stessa risposta del proprietario, ma ne ha il diritto. Il verdetto finale spetta quindi a una matrice dei privilegi, che assegna a ogni ruolo il rango dichiarato nel file di configurazione. L'accesso da un rango superiore a uno inferiore è legittimo; quello fra ranghi uguali è una BOLA orizzontale; quello da un rango inferiore a uno superiore è una BOLA verticale. Un ruolo assente dalla configurazione viene trattato come il più basso, e il modulo emette un avviso. Ogni esito diventa una segnalazione di runtime: una vulnerabilità confermata, con severità HIGH, oppure un esito sicuro, con severità informativa. La richiesta anonima serve infine a un controllo in più: se la risorsa viene restituita anche senza credenziali, il modulo segnala un difetto di autenticazione sullo stesso endpoint.

Conferma e smentita Il motore di correlazione usa la chiave comune, alla quale è legata anche la segnalazione di Spectral, per riunire in un'unica voce l'ipotesi statica

e gli esiti di runtime. Conferme e smentite, però, sono trattate in modo diverso. Se l'attacco riesce, la voce conserva la segnalazione statica, che indica la riga da correggere, ne porta la severità a `CRITICAL` e la confidenza a 1, e allega l'esito dinamico come evidenza. Se l'attacco viene bloccato, la severità resta invariata, la confidenza viene dimezzata e la voce è contrassegnata come smentita, senza evidenza di runtime; il dimezzamento avviene una sola volta, anche quando le smentite sono più di una. Se sulla stessa chiave arrivano sia una conferma sia una smentita, prevale la conferma, qualunque sia l'ordine di arrivo.

L'analisi statica indica dove potrebbe trovarsi il difetto e raggiunge anche endpoint che il livello dinamico non vede. La verifica dinamica stabilisce se il difetto è reale e copre i controlli che sono presenti ma sbagliati.

3.3.2 API2 – L'identità del chiamante e la validità dei token

Nella *Broken Authentication* [6] il difetto riguarda la capacità del sistema di stabilire chi sia il chiamante. Dal codice il modulo ricostruisce un'idea del sistema di autenticazione, attraverso l'analisi delle librerie adottate, le funzioni di accesso, i middleware e i decorator a protezione delle rotte e i claim inclusi nei token. Il codice non dice però come il meccanismo reagisce: se rifiuta un token falso o scaduto, se chiude la sessione dopo il logout, se frena i tentativi di accesso ripetuti. La divisione ricalca quella della letteratura: le debolezze che riguardano la firma e la validazione dei token dipendono da regole deterministiche e si prestano sia all'ispezione del codice sia alla generazione di token manipolati [39, 40], mentre l'assenza di freni sui tentativi di accesso emerge dall'osservazione del traffico più che da un'ispezione sintattica [7].

Il grafo di conoscenza dell'autenticazione Il modulo non presuppone di conoscere il bersaglio: ne ricava lo stack e la gestione dell'identità da tre fonti, fuse in un unico modello strutturato, il *grafo di conoscenza dell'autenticazione*. La prima è una ricognizione dei file dichiarativi, cioè manifest delle dipendenze (da `package.json` a `requirements.txt` a `pom.xml`), file di configurazione e di infrastruttura, sui quali euristiche deterministiche riconoscono lo stack applicativo, le librerie e i proto-

colli usati per le credenziali. Le stesse informazioni vengono chieste in parallelo a un modello linguistico eseguito localmente tramite Ollama, che restituisce un oggetto strutturato; le euristiche hanno però la precedenza, perché il risultato resti riproducibile a parità di input, e se il modello è irraggiungibile l'analisi prosegue sul solo esito delle euristiche. La fase è statica e offline e non interroga il bersaglio in esecuzione. La seconda fonte è il codice sorgente, da cui l'analizzatore estrae rotte di accesso, middleware di protezione e invocazioni delle librerie dedicate; a ogni file assegna poi un punteggio in base a quanti di questi segnali contiene e trattiene solo i file oltre una soglia, così da concentrare le fasi seguenti su quelli che governano davvero l'identità. Quando è cablato nel codice, recupera anche il segreto usato per firmare i token, un dettaglio che diventa decisivo nella fase dinamica.

La terza fonte entra in gioco nella fase di correlazione del modulo, che fonde i segnali statici con la specifica OpenAPI, da cui ricava gli schemi di sicurezza dichiarati e le rotte protette, e con l'eventuale traffico catturato a runtime, di cui decodifica i token per estrarne claim, ruoli e permessi. Il grafo che ne risulta sintetizza schemi di autenticazione, endpoint protetti e relativi controlli, e porta con sé un grado di confidenza fra 0 e 1 che parte da una base minima e cresce con ogni fonte concorde: l'analisi del codice, la specifica, il traffico e il riconoscimento di un provider noto. È la correlazione fra livelli di RQ_{1.1} tradotta in un numero: nessuna fonte basta da sola a fissare il modello, ma la loro convergenza lo rende più affidabile, e più avanti questo valore decide quali prove abbiano fondamento sufficiente per essere tentate.

La batteria di prove Ricostruito il modello, il modulo sollecita l'applicazione con richieste reali. Dopo un controllo di raggiungibilità e la localizzazione degli endpoint di autenticazione esegue una batteria di prove asincrone, ciascuna dedicata a una debolezza distinta e conclusa con un esito `PASS`, `FAIL` o `INCONCLUSIVE`, con severità e categoria interna. La Tabella 3.3 le raggruppa per l'aspetto dell'autenticazione che mettono alla prova.

Il legame più stretto con la parte statica passa per il segreto di firma. Quando è disponibile, la prova sul token scaduto non invia un token qualsiasi, ma ne costruisce uno crittograficamente valido con scadenza nel passato e osserva se il backend lo accetta, distinguendo così un difetto reale di validazione da un semplice rifiuto per

Aspetto	Prova	Debolezza sollecitata
Firma e validazione dei token	T01	token manipolato accettato
	T02	token scaduto accettato
	T07	confusione di algoritmo (RS256→HS256)
	T17	iniezione di header JWKS
Gestione di sessione e token	T05	token ancora valido dopo il logout
	T06	session fixation
	T12	refresh token riutilizzabile
	T13	refresh token non ruotato
	T14	refresh token a durata illimitata
	T15	refresh token usato in parallelo
Flussi e credenziali	T04	token riusato su un'altra audience
	T08	escalation di privilegio tramite i claim
	T18	flussi OAuth2/OIDC senza protezioni (state, PKCE)
	T19	credenziali non basate su token
Forza bruta e limitazione	T03	login senza limite ai tentativi
	T20	limite aggirato con lo spoofing dell'header IP
Autenticazione a più fattori e SSO	T21	MFA aggirabile od OTP debole
	T22	firma delle asserzioni SAML manomessa
Comportamento e riservatezza	T09	cookie senza flag di sicurezza
	T10	informazioni sensibili nei messaggi di errore
	T11	enumerazione degli utenti
	T16	reset della password debole

Tabella 3.3: Le prove dinamiche del modulo, una per riga e raggruppate per aspetto dell'autenticazione.

firma non riconosciuta. Le prove che presuppongono una sessione autenticata vengono tentate solo se la confidenza del modello supera una soglia predefinita, per non produrre esiti privi di fondamento quando la ricostruzione è incerta, e l'esecuzione resta entro guardrail non distruttivi: la prova di forza bruta, per esempio, viene omessa se il bersaglio è dichiarato di produzione. Se il bersaglio non espone un proprio endpoint di login, perché delega il riconoscimento a un provider esterno, le prove che richiederebbero un token non falliscono ma si dichiarano non conclusivi, tenendo separata l'assenza di una debolezza dall'impossibilità di verificarla.

Nel punteggio di robustezza ogni prova fallita sottrae una quota proporzionale al proprio peso, e a ciascuna il motore di remediation della Sottosezione 3.2.3 affianca un'indicazione correttiva, tratta dalla base di conoscenza locale o, in sua assenza, generata dal modello linguistico locale.

La regola statica S01 Proprio quando l'autenticazione è delegata a un provider esterno la verifica dinamica ha meno presa, perché senza un login del bersaglio i token non si possono mettere alla prova. Per questo caso esiste la regola S01, puramente statica, che individua una funzione che scompone il token nei tre segmenti separati da punti, decodifica il payload in Base64 e lo interpreta come JSON senza mai invocare una libreria di verifica della firma. È un ripiego insicuro tipico: in mancanza della validazione crittografica il codice si fida comunque del contenuto del token, e un attaccante può forgiarne uno con qualsiasi identità. La lettura non sicura del payload resta visibile nel sorgente anche quando il sistema non è raggiungibile o non offre un punto d'ingresso da provare: la componente statica della categoria rimane rilevabile proprio dove quella dinamica non lo è.

3.3.3 API3 – Le proprietà protette di un oggetto

La *Broken Object Property Level Authorization* [41] (Sottosezione 2.2.3) ha due direzioni: una proprietà riservata restituita a chi non dovrebbe vederla e una proprietà interna accettata in scrittura da chi non dovrebbe modificarla. Codice e contratto indicano quali proprietà l'applicazione considera riservate e dove rischia di esporle, mentre solo il sistema in esecuzione dice se la protezione regge. La difficoltà propria di questa categoria sta però a monte. Il criterio impedisce di fornire al modulo un elenco di campi sensibili, che del resto funzionerebbe sulle applicazioni di prova e su nessun altro sistema, e prima di cercare dove una proprietà possa sfuggire il modulo deve quindi dedurre da sé quali proprietà siano protette.

Inventario e proprietà protette L'inventario degli oggetti e delle loro proprietà nasce da due fonti. Dal codice, in Python, JavaScript, TypeScript e Go, il modulo estrae

le classi che modellano i dati, come gli oggetti di trasporto¹ e le entità di persistenza; dal contratto OpenAPI legge gli schemi di richiesta e risposta di ciascuna operazione. Per confrontare le due fonti, i nomi vengono ricondotti a una forma comune: le classi `UserDTO` o `UserEntity` nel codice e il percorso `/users` nel contratto diventano entrambi `User`.

Una proprietà è considerata protetta quando l'applicazione stessa la tratta come tale. Il modulo cerca nel codice i punti che solo un utente con un certo ruolo può raggiungere, come una funzione riservata all'amministratore o il ramo di una condizione sul ruolo del chiamante, e considera protette le proprietà lette o modificate lì. Il riconoscimento non si basa sui nomi delle proprietà ma sui modi in cui i framework più diffusi dichiarano un controllo di ruolo, raccolti in un catalogo di configurazione estendibile. Come anticipato in apertura di sezione, non contano né la verifica che chiede soltanto di aver effettuato l'accesso, che tutti gli utenti autenticati superano vedendo gli stessi campi, né il controllo sul proprietario, che riguarda l'oggetto e non le sue proprietà. Il contratto aggiunge un secondo indizio, dotato di una direzione: un campo di sola lettura è protetto in scrittura, uno di sola scrittura, come una password, in lettura. Il modulo conserva questa direzione, per cui un campo di sola lettura che compare in una risposta non è un difetto.

Regole statiche e verifica per proprietà Individuate le proprietà protette, quattro regole statiche cercano i punti in cui la protezione può essere aggirata (Tabella 3.4). La regola S01 cerca il *mass assignment*², cioè il corpo della richiesta copiato per intero su un oggetto, per esempio con un ciclo su tutti i campi ricevuti. Se il codice accetta solo i campi di un'*allowlist*³ la segnalazione cade; se ne esclude alcuni con una *denylist* resta, perché ogni altro campo rimane scrivibile. La regola S02 cerca il caso opposto,

¹*DTO (Data Transfer Object)*: oggetto usato per trasferire dati fra il client e il server, spesso una proiezione parziale dell'entità memorizzata nel database.

²*Mass assignment*: l'assegnazione automatica dei campi ricevuti dal client agli attributi di un oggetto applicativo; se il server non filtra i campi ammessi, il client può valorizzare attributi interni non previsti dal flusso normale.

³*Allowlist* e *denylist*: elenco dei valori ammessi e, rispettivamente, dei valori esclusi. Filtrare i campi con un'*allowlist* blocca tutto ciò che non è previsto; una *denylist* lascia passare qualunque campo non elencato, compresi quelli aggiunti in seguito.

un oggetto con proprietà protette serializzato e restituito così com'è fuori da un controllo di ruolo. Le regole S03 e S04 guardano il contratto: la prima segnala un campo di sola lettura che un'altra operazione accetta in scrittura, o di sola scrittura che compare in una risposta; la seconda una proprietà protetta che circola nel traffico ma che il contratto non documenta.

Regola	Direzione	Cosa segnala
S01	scrittura	corpo della richiesta copiato per intero su un oggetto, senza allowlist dei campi
S02	lettura	oggetto con proprietà protette restituito per intero, fuori da un controllo di ruolo
S03	entrambe	campo di sola lettura accettato in scrittura, o di sola scrittura restituito
S04	entrambe	proprietà protetta presente nel traffico ma assente dal contratto

Tabella 3.4: Le quattro regole statiche del modulo. Ogni segnalazione nasce “da verificare” e viene confermata o smentita dalle prove dinamiche della stessa direzione.

La verifica dinamica usa le credenziali di un utente ordinario e di un amministratore e agisce sempre sulla risorsa dell'utente ordinario, per esempio il suo profilo: l'accesso all'oggetto è quindi legittimo, e l'unico controllo messo alla prova è quello sulle proprietà. Le due direzioni vanno provate separatamente, perché un'API può accettare in scrittura una proprietà che non restituisce mai, oppure restituire dati riservati senza consentirne la modifica. Le prove sono sette. In lettura verificano se l'utente ordinario riceve una proprietà protetta e se la riceve esattamente come l'amministratore. In scrittura verificano se riesce a modificare una proprietà protetta, ad aggiungere alla richiesta campi che il client normalmente non invia (mass assignment) o proprietà non documentate dal contratto, a modificare una proprietà che non viene mai restituita e a far prevalere la propria scrittura su quella dell'amministratore.

Una prova di lettura si giudica dalla risposta, una di scrittura no, perché molti server rispondono con successo e scartano in silenzio i campi non ammessi: il modulo scrive quindi un valore riconoscibile, rilegge la risorsa come amministratore e conferma il difetto solo se quel valore risulta salvato. Gli esiti tornano poi sulle segnalazioni statiche, che diventano *confermate* se una prova della stessa direzione ha successo su una delle loro proprietà e *non confermate* se la protezione ha retto. Nel punteggio di robustezza ogni tipo di prova riuscita sottrae la stessa quota.

3.3.4 API5 – Il ruolo richiesto per invocare una funzione

La *Broken Function Level Authorization* [19] (Sottosezione 2.2.5) riguarda le funzioni che un ruolo non dovrebbe poter invocare, come esportare l'elenco degli utenti, cancellare un account altrui o promuovere sé stessi ad amministratore. Rispetto alla BOLA la domanda non è più se un utente possa accedere a un certo oggetto, ma se possa eseguire una certa funzione. Il difetto nasce spesso da una dimenticanza: una rotta aggiunta a un controller eredita il routing e l'autenticazione, ma non l'autorizzazione, e resta raggiungibile da chiunque abbia effettuato l'accesso. L'implementazione riconosce la mancanza del controllo sul ruolo del chiamante in una funzione riservata. Per stabilire se l'assenza sia davvero sfruttabile servirebbe invocare la funzione con le credenziali di un utente ordinario, una verifica dinamica che il modulo non compie. Il rilevatore è perciò interamente statico e segue lo schema comune, al quale aggiunge un livello dedicato ai file di configurazione e una regola che segue il flusso dei dati.

Funzioni privilegiate e controlli di ruolo Sulla mappa degli endpoint, un endpoint è considerato privilegiato quando il suo percorso, con i parametri sostituiti da un segnaposto, corrisponde a uno schema tipico delle funzioni riservate, come `/admin`, `/internal`, `/debug`, `/config`, `/export` o `/users/{id}/role`, oppure quando è una cancellazione o una sostituzione rivolta a un'intera collezione anziché a un singolo elemento. Per i controlli, il modulo tiene due elenchi separati di decoratori e middleware: quelli che, come `login_required`, verificano *chi* è il chiamante e quelli che, come `require_role("admin")`, verificano *che cosa* può fare. Solo i secondi sono controlli di funzione, e il modulo li riconosce anche quando sono scritti nel corpo della funzione, come un confronto sul ruolo dell'utente corrente.

Le regole di questo livello sono sei. La più grave, BF-001, segnala un endpoint privilegiato privo di qualsiasi controllo di ruolo (CWE-285 [20]). BF-002 riguarda una funzione protetta solo dal login che compie un'operazione sensibile, come una cancellazione, un cambio di ruolo o un rimborso, e resta alla portata di qualunque utente registrato (CWE-862 [42]). BF-003 riguarda il verbo HTTP⁴ (CWE-650 [43]): una

⁴Verbo (o metodo) HTTP: l'azione richiesta su una risorsa, per esempio GET per leggere o DELETE

rotta privilegiata che accetta qualsiasi metodo, oppure un'applicazione che rispetta l'intestazione `X-HTTP-Method-Override` senza limitarne i valori, permette di raggiungere l'operazione per una via diversa da quella prevista. BF-004 segnala le aree amministrative esposte (CWE-284 [44]), cioè un raggruppamento di rotte con prefisso `/admin` privo di un controllo di ruolo comune, oppure una funzione che restituisce dati in blocco, come l'esportazione di tutti gli utenti, collocata su un percorso ordinario. BF-006 cerca le funzioni *ombra*, cioè endpoint di debug, di test o di popolamento dei dati rimasti nel codice di produzione (CWE-285), con una severità alta se l'endpoint è del tutto aperto, media se richiede solo il login e bassa se è protetto ma resta un residuo da rimuovere. L'ultima regola del livello, BF-007, segue il flusso dei dati ed è descritta più avanti.

Configurazione e contratto Nei file di configurazione il modulo cerca il *diniego per impostazione predefinita*⁵, che il capitolo precedente ha indicato come difesa centrale della categoria. La regola BF-005 (CWE-276 [45]) scatta quando un'applicazione Django REST Framework non dichiara una classe di permessi predefinita o dichiara quella che consente tutto, quando un'applicazione FastAPI non registra dipendenze globali e quando un'applicazione Express non monta un middleware di autenticazione prima dei router.

Nel contratto il modulo segnala le operazioni privilegiate prive di uno schema di sicurezza⁶, locale o globale, e i percorsi amministrativi esposti. Il confronto con il codice produce due casi propri di questo modulo. Un'operazione dichiarata nella specifica e accettata dal codice con qualsiasi verbo produce una segnalazione BF-003 con l'evidenza di entrambi i livelli. Un endpoint di debug o di test presente nel codice e assente dal contratto è invece il caso più affidabile di BF-006, perché l'assenza dal contratto conferma che quella funzione non doveva essere pubblica. La Tabella 3.5 riepiloga le sette regole.

per cancellare.

⁵*Deny-by-default*: il criterio per cui ogni funzione è negata salvo concessione esplicita al ruolo che ne ha diritto; con il criterio opposto una dimenticanza non produce alcun errore e passa inosservata.

⁶*Schema di sicurezza (security scheme)*: la sezione del contratto OpenAPI che dichiara come un endpoint è protetto, per esempio tramite token o chiave API.

Regola	CWE	Debolezza ricercata	Severità
BF-001	285	endpoint privilegiato privo di controllo di ruolo	CRITICAL
BF-002	862	operazione sensibile protetta solo dall'autenticazione	HIGH
BF-003	650	rotta privilegiata raggiungibile con qualsiasi verbo HTTP	HIGH
BF-004	284	area amministrativa o funzione in blocco esposta	HIGH
BF-005	276	permesso predefinito anziché diniego (configurazione)	MEDIUM
BF-006	285	funzione di debug o di test rimasta esposta	HIGH-LOW
BF-007	269	ruolo scelto dal chiamante nel corpo della richiesta	CRITICAL

Tabella 3.5: Le sette regole del modulo. BF-005 opera sui file di configurazione; BF-001, BF-003, BF-004 e BF-006 sul codice e sul contratto OpenAPI; BF-002 e BF-007 sul solo codice.

L'escalation di privilegio tramite il corpo della richiesta Alcuni difetti non si vedono in un singolo punto, come un decoratore, un percorso o una voce di configurazione, ma nel percorso che un dato compie all'interno della funzione. Il caso tipico è una funzione che crea un invito o aggiorna un profilo e prende il ruolo da assegnare direttamente dal corpo della richiesta: basta inviare `{"role": "admin"}` per ottenere privilegi che nessuno ha concesso. La regola BF-007 (CWE-269 [46]) è stata introdotta per questo scenario ed è la prima del modulo a seguire un flusso di dati. L'analisi resta all'interno della singola funzione e ne percorre le istruzioni nell'ordine in cui sono scritte, a partire dalle *sorgenti* del contenuto della richiesta: in Python il corpo JSON o i campi del modulo, oltre ai parametri FastAPI tipizzati con un modello di dati, in Express il corpo della richiesta, anche quando viene scomposto in variabili. Se una variabile riceve il corpo e da essa si legge il campo del ruolo, anche la variabile che lo riceve è considerata controllata dal chiamante, finché un'assegnazione successiva di un valore fisso non interrompe il legame. I campi seguiti sono

quelli che esprimono un privilegio, come `role`, `is_admin`, `permissions`, `group` o `scope`, in un elenco configurabile.

La regola scatta quando uno di questi valori raggiunge un'*assegnazione*: un attributo di un oggetto, un argomento passato alla creazione o all'aggiornamento di un record, un dizionario salvato nel database o una clausola SQL `UPDATE`. Per contenere i falsi positivi sono esclusi gli usi in sola lettura, perché un ruolo passato a un filtro o a una ricerca serve a selezionare dati e non ad assegnarli, e la segnalazione viene soppressa se la funzione verifica il privilegio del chiamante prima dell'assegnazione, con un decoratore di ruolo o con una condizione che interroga il ruolo dell'utente corrente. La semplice autenticazione, anche qui, non basta.

Deduplicazione per priorità di regola Nella BFLA le segnalazioni sono raggruppate per solo percorso, così che lo stesso endpoint rilevato dal codice e dal contratto confluisca in una voce unica, e l'ordine di priorità è

BF-006 > BF-004 > BF-003 > BF-002 > BF-001 > BF-005.

Dire che un endpoint è una funzione di debug dimenticata, o che si trova in un'area amministrativa senza protezione comune, spiega il difetto meglio del generico "manca un controllo di ruolo" e ne indica anche il rimedio. A parità di priorità prevale la segnalazione ricavata dal codice, che punta a una riga precisa, su quella ricavata dal solo contratto, mentre le segnalazioni di configurazione, che non si riferiscono a un endpoint, sono sempre conservate.

Come OA-005 nel modulo della BOLA, BF-007 resta fuori dalla competizione: le altre regole chiedono se il chiamante possa invocare la funzione, BF-007 se possa scegliere il valore che la funzione assegna. Su un endpoint di cambio ruolo privo di controllo di ruolo, per esempio, le due segnalazioni convivono, BF-002 per indicare che la funzione è accessibile a qualsiasi utente autenticato e BF-007 per indicare che il nuovo ruolo lo decide chi invia la richiesta, e sopprimerne una nasconderebbe metà del problema.

3.4 Categorie tecniche: un metodo per categoria

Le categorie raccolte in questa sezione sono quattro: l'Unrestricted Resource Consumption (API4), la Server-Side Request Forgery (API7), la Security Misconfiguration (API8) e l'Unsafe Consumption of APIs (API10). Secondo il criterio della Sottosezione 3.1.3 si comportano come categorie tecniche: la violazione si riconosce leggendo l'elaborato di progetto, senza sollecitare il sistema in esecuzione, e i rilevatori sono statici. È su di esse che il sistema cerca una risposta a RQ_{1.1}.

A differenza delle categorie ibride, però, non hanno una forma comune da descrivere una volta sola. Ciascuna richiede un metodo proprio: per il consumo di risorse un limite va cercato su tre livelli e ne vanno confrontati i risultati, per la SSRF conta il percorso del dato dall'input dell'utente alla chiamata di rete, per la configurazione il confronto con valori di riferimento noti, per il consumo di API esterne il modo in cui il codice tratta le risposte che riceve. Per questo le quattro categorie sono descritte separatamente.

3.4.1 API4 – I limiti sul consumo di risorse, su tre livelli

Ogni richiesta a un'API ha un costo: tempo di calcolo, memoria, banda, a volte denaro vero, come quando il server invia un SMS o interroga un modello di intelligenza artificiale a pagamento. L'*Unrestricted Resource Consumption* [11] (Sottosezione 2.2.4) riguarda le API che non mettono un tetto a questo costo. In quei casi basta una richiesta che chiede un milione di risultati, un file enorme o mille SMS di fila per rallentare il servizio o far lievitare la bolletta.

Rispetto alle categorie di autorizzazione viste finora c'è una differenza importante. L'assenza di un limite può essere individuata direttamente attraverso l'analisi statica del progetto, verificando nel codice e nella configurazione se sono presenti meccanismi come rate limiting, timeout o limiti di paginazione, senza dover eseguire o attaccare il sistema. Per questo motivo la categoria rientra fra quelle *tecniche* della Sottosezione 3.1.3 e il modulo si basa esclusivamente sul rilevamento statico.

Lo stesso limite, però, può essere definito in punti diversi del sistema. Un controllo sulla dimensione dei file può essere presente nel codice dell'applicazione, imposto dal proxy che la precede oppure dichiarato nel contratto dell'API. La sua assenza

dal codice, quindi, non è sufficiente per stabilire la presenza della vulnerabilità: il limite potrebbe infatti essere applicato a un livello esterno all'applicazione. Il modulo analizza staticamente tutti e tre i livelli e ne confronta i risultati, come previsto dalla RQ_{1.1}. In questo caso la correlazione tra i livelli non rappresenta un semplice raffinamento dell'analisi, ma è necessaria per distinguere un'assenza locale da una reale assenza del limite nel sistema.

I tre livelli di analisi Nel codice sorgente, Python e JavaScript, il modulo cerca i punti in cui un limite servirebbe e manca. Qualche esempio: una paginazione⁷ in cui l'utente può alzare a piacere il numero di risultati; un file caricato senza guardarne la dimensione; una chiamata verso un altro servizio priva di timeout⁸, che può restare appesa a lungo; un ciclo che scorre un elenco inviato dal client senza controllarne la lunghezza; un servizio a pagamento che chiunque può invocare quante volte vuole.

Per le API GraphQL, che permettono al client di costruire query specificando i dati e le relazioni da restituire, verifica anche che le query non possano raggiungere una profondità di annidamento eccessiva, evitando così richieste capaci di consumare una quantità elevata di risorse.

Nella configurazione controlla che i container abbiano un tetto di memoria, che il proxy o l'applicazione fissino una dimensione massima per le richieste e che i server definiscano un timeout per quelle troppo lente. Un timeout impostato a zero, che di fatto spegne il controllo, pesa più di uno semplicemente dimenticato.

Il contratto OpenAPI, quando c'è, descrive ciò che l'API promette ai suoi client. Qui il modulo verifica che i parametri di paginazione abbiano un valore massimo, che i caricamenti abbiano una dimensione massima⁹, che le operazioni *batch*¹⁰ fissino quanti elementi possono ricevere e che gli endpoint costosi siano protetti.

⁷*Paginazione*: la suddivisione di una risposta voluminosa in pagine di dimensione limitata, così che una singola richiesta non restituisca un'intera collezione.

⁸*Timeout*: il tempo massimo concesso a un'operazione prima di interromperla; in sua assenza una chiamata lenta può restare appesa a tempo indefinito, trattenendo risorse.

⁹Nel contratto OpenAPI i vincoli *maximum*, *maxLength* e *maxItems* dichiarano rispettivamente il valore, la lunghezza e il numero di elementi massimi ammessi per un campo.

¹⁰*Operazione batch*: una singola richiesta che trasporta un elenco di elementi da elaborare insieme.

Capire quali endpoint siano costosi è la parte più delicata e rimane necessariamente un'euristica. Il nome aiuta, perché `/sms` o `/export` fanno pensare a un lavoro pesante, ma da solo può ingannare: leggere una fattura già emessa non costa nulla, emetterla sì. Il modulo guarda perciò anche a cosa fa l'operazione e a cosa riceve. Un numero di telefono accompagnato da un testo, un importo con la sua valuta o un *prompt*, cioè una richiesta inviata a un modello di intelligenza artificiale, sono indizi più affidabili del solo nome dell'endpoint. Questa valutazione resta comunque basata su euristiche e può quindi produrre casi che richiedono una verifica ulteriore.

Ogni segnalazione indica la regola violata, la debolezza CWE, la severità e la difesa che ci si aspettava di trovare, così chi legge sa subito cosa aggiungere.

L'ultimo passo mette a confronto i tre livelli. Il modulo risale da ogni segnalazione nel codice alla rotta che la contiene e la accosta all'operazione corrispondente del contratto. Quando lo stesso limite manca in entrambi, come un parametro `limit` senza tetto nel codice e nel contratto, le due segnalazioni vengono collegate e aumenta la confidenza associata alla rilevazione. Quando invece un altro livello copre il buco, per esempio un proxy che rifiuta i file troppo grandi, la segnalazione resta nel rapporto ma la sua severità viene impostata a `INFO`, con il riferimento alla difesa che la compensa.

Il codice, infine, può chiarire un dubbio che il contratto lascia aperto. Se una rotta chiama un servizio a pagamento è costosa anche quando ha un nome innocuo, e il modulo la segnala se il contratto non prevede alcuna protezione.

3.4.2 API7 – Dall'input dell'utente alla chiamata di rete

La *Server-Side Request Forgery* [24] (Sottosezione 2.2.7) si ha quando un endpoint usa un dato fornito dall'utente per decidere verso quale indirizzo il server debba inviare una richiesta, senza controllare la destinazione. Un servizio che scarica l'immagine di profilo da un URL indicato dall'utente, per esempio, può essere indotto a interrogare indirizzi della rete interna, che dall'esterno non sarebbero raggiungibili. Il caso più grave riguarda il servizio di metadati delle macchine cloud, un indirizzo interno da cui ogni istanza legge la propria configurazione e, spesso, le proprie credenziali temporanee: chi riesce a farlo interrogare dal server può impossessarsene.

Il modulo è statico, e le fonti primarie sostengono questa scelta. OWASP giudica facile la rilevabilità della categoria, con la sola eccezione della forma cieca, in cui il server invia la richiesta ma non ne restituisce la risposta [24]; la CWE-918 indica come metodo più efficace l'analisi statica automatica [25]. Il motivo è che qui la domanda è una proprietà del codice: un dato dell'utente arriva o non arriva a una chiamata di rete, e lo si può stabilire senza conoscere le intenzioni di chi ha scritto il programma. Gli studi sull'argomento ne mostrano anche i limiti, dalla necessità di verificare a mano i risultati [47] ai percorsi segnalati ma non sfruttabili [48], fino al ricorso a tecniche dinamiche [49]. Per il modulo ne derivano tre difficoltà: conoscere tutte le funzioni che inviano richieste di rete, non segnalare i dati dell'utente che finiscono in una richiesta senza influire sull'indirizzo, e riconoscere i controlli sull'URL posti lungo il percorso. I paragrafi che seguono mostrano come vengono affrontate.

Il flusso dei dati Il modulo usa Semgrep, uno strumento di analisi statica che cerca nel codice schemi descritti da regole, in modalità di *analisi della contaminazione*¹¹. Le sorgenti sono i punti in cui entra l'input dell'utente: parametri della query, corpo della richiesta, campi di un modulo, intestazioni HTTP e, nelle applicazioni Node.js, parametri del percorso. I sink sono le funzioni dei principali client HTTP di Python e di JavaScript. La segnalazione scatta solo quando il dato contaminato finisce nell'indirizzo della richiesta, non in un altro argomento, e questo risponde alla seconda difficoltà. L'indirizzo può essere anche solo in parte controllato dall'utente, per esempio quando l'input viene concatenato a un URL di base fisso: anche così può spostare la richiesta su un percorso diverso da quello previsto, e il modulo segue il dato attraverso concatenazioni e formattazioni di stringhe.

La terza difficoltà riguarda i controlli. Il flusso si interrompe quando il dato passa per una funzione di validazione dell'URL o per un controllo che verifica che la destinazione appartenga a un elenco di host autorizzati, e la chiamata protetta da

¹¹*Analisi della contaminazione (taint analysis)*: tecnica che marca come “contaminati” i valori provenienti da una *sorgente* esterna, cioè il punto in cui un dato controllato dall'esterno entra nel programma, e ne segue la propagazione fino a un *sink*, l'operazione sensibile in cui il valore viene infine usato.

quel controllo non viene segnalata. La prima difficoltà, infine, si traduce nell’elenco delle regole, che coprono i client HTTP più diffusi nei due linguaggi (Tabella 3.6).

Regola	Cosa segnala	Livello
SS-001	dato dell’utente nell’URL di una chiamata HTTP, in Python	codice
SS-002	lo stesso, con le funzioni di rete della libreria standard di Python	codice
SS-003	lo stesso, nelle applicazioni Node.js	codice
SS-004	chiamata con URL dell’utente che segue le redirezioni in automatico	codice
SS-005	chiamata verso l’indirizzo del servizio di metadati cloud	codice
SS-006	parametro che rappresenta un URL senza vincoli sul formato o sulle destinazioni ammesse	contratto

Tabella 3.6: Le sei regole del modulo: cinque sul codice sorgente e una sul contratto OpenAPI.

Due regole guardano oltre il semplice flusso. SS-004 segnala le chiamate che seguono le redirezioni in automatico, perché un indirizzo autorizzato può rimandare il server verso uno interno dopo che il controllo è già stato superato. SS-005 non segue un flusso, ma cerca nel codice gli indirizzi dei servizi di metadati dei principali provider cloud.

Il contratto SS-006 lavora sulla specifica OpenAPI. Individua i parametri e i campi del corpo che rappresentano una destinazione, come `webhook` o `callback_url`, e verifica che il contratto ne vincoli il valore: un formato URL dichiarato (`format: uri`) e un’espressione regolare che limiti le destinazioni ammesse. Se mancano entrambi la segnalazione è grave, se manca solo la seconda è lieve. Il contratto non dice se l’implementazione sia vulnerabile, ma mostra se l’interfaccia lascia aperta la porta già in fase di progetto.

Resta fuori dalla portata del modulo la forma cieca, l’unica che OWASP considera difficile da rilevare: accertarla richiede di osservare le richieste che il server invia davvero, e quindi un sistema in esecuzione.

3.4.3 API8 – La configurazione confrontata con i valori di riferimento

La *Security Misconfiguration* [21] (Sottosezione 2.2.8) raccoglie errori di configurazione che possono verificarsi a diversi livelli, dalla rete fino all’applicazione. Questa

sottosezione ne affronta la parte visibile nel codice sorgente e nei file di progetto, completando l'analisi infrastrutturale già trattata nella Sezione 3.2. Il modulo adotta un approccio statico: cerca direttamente nel codice le scelte di configurazione insicure, senza richiedere l'avvio o l'esecuzione del software.

Qui il rilevamento statico parte avvantaggiato. Nelle categorie di autorizzazione uno strumento non sa se un controllo fosse dovuto, e per il consumo di risorse non sa se un limite sia applicato altrove. Per la configurazione, invece, esiste un riferimento pubblico: valori raccomandati, impostazioni predefinite note per essere insicure, versioni di librerie con vulnerabilità note. Basta confrontare il progetto con quel riferimento, senza dover dedurre le intenzioni di chi lo ha scritto. Per questo OWASP giudica facile la rilevabilità della categoria e rinvia agli strumenti automatici [21].

Regole locali e regole globali Il modulo analizza la struttura sintattica dei file Python ed esegue controlli testuali su quelli JavaScript e TypeScript, escludendo le directory non applicative (come ambienti virtuali o cartelle di build). I controlli sono articolati in verifiche locali su ciascun file e verifiche globali sul progetto (Tabella 3.7).

A livello di singolo file, il modulo intercetta i comportamenti a rischio più comuni: politiche CORS¹² permissive o aperte assieme alle credenziali utente, modalità di debug e gestori di errore che espongono la traccia dello stack¹³, segreti cablati nel codice individuati tramite entropia di Shannon¹⁴ e rotte prive di restrizioni sui metodi HTTP o dell'intestazione `Cache-Control`¹⁵.

A livello globale, tre verifiche valutano l'impostazione complessiva del servizio: l'assenza delle intestazioni HTTP difensive raccomandate (come HSTS e `Content-Security-Policy`), la totale mancanza di una politica CORS e la pre-

¹²CORS (*Cross-Origin Resource Sharing*): meccanismo con cui un server dichiara quali siti di origine diversa possono leggere le sue risposte dal browser; una configurazione permissiva consente a domini terzi non autorizzati di accedere a dati riservati.

¹³Traccia dello stack (*stack trace*): l'elenco delle chiamate di funzione attive nel momento dell'errore; mostrata al client rivela percorsi, librerie e struttura interna dell'applicazione.

¹⁴Entropia di Shannon: misura del grado di casualità di una stringa; sequenze ad alto disordine indicano chiavi o token reali, distinguendoli dai normali identificatori.

¹⁵`Cache-Control`: intestazione HTTP che indica a browser e proxy se e per quanto tempo possono conservare una copia della risposta; se manca, dati riservati possono restare in una cache condivisa.

senza di dipendenze con vulnerabilità note, individuate confrontando le versioni dichiarate con un elenco di versioni vulnerabili. Ogni segnalazione porta con sé un grado di confidenza, e quelle sotto la soglia di 0,70 vengono scartate prima di comporre il rapporto.

Regola	Comportamento ricercato	Ambito	Severità
SC-001	politica CORS con carattere jolly	file	HIGH
SC-002	modalità di debug attiva	file	HIGH
SC-003	gestore d'errore che espone la traccia dello stack	file	HIGH
SC-004	header di sicurezza mancanti	globale	MEDIUM
SC-005	segreto cablato nel codice (entropia di Shannon)	file	CRITICAL
SC-006	assenza totale di politica CORS applicativa	globale	MEDIUM
SC-007	dipendenze applicative con vulnerabilità note	globale	HIGH
SC-008	metodi HTTP non ristretti o operazioni che modificano dati esposte tramite GET	file	MEDIUM
SC-009	assenza di Cache-Control su rotte con dati sensibili	file	MEDIUM

Tabella 3.7: Il catalogo delle regole del modulo di rilevamento applicativo per API8:2023.

3.4.4 API10 – La fiducia nelle risposte dei servizi esterni

Nella maggior parte delle categorie viste finora il pericolo arriva dai dati che l'API riceve dai suoi utenti. *L'Unsafe Consumption of APIs* [29] (Sottosezione 2.2.10) ribalta il punto di vista: qui il pericolo arriva dai servizi di terze parti da cui l'applicazione stessa dipende, come un gateway di pagamento o un fornitore di dati esterni. Avendoli scelti e configurati, l'applicazione tende a fidarsi delle loro risposte senza troppe

domande; ma quelle risposte attraversano pur sempre un confine di fiducia¹⁶ che spesso nessuno sorveglia. OWASP elenca cinque forme di questa fiducia mal riposta: canale non cifrato, dati non validati, redirezioni seguite ciecamente, risorse senza limiti e timeout assenti. Nel repository del progetto OWASP è stato segnalato che alcune di queste forme si sovrappongono ad altre categorie della stessa lista [50].

Il modulo si occupa di tre forme (canale in chiaro, dati non validati e redirezioni cieche) e lascia le altre due, l'assenza di limiti e di timeout, al modulo dell'Unrestricted Resource Consumption (Sottosezione 3.4.1), che ne è la sede naturale; così lo stesso difetto non viene contato due volte, come chiede il criterio della Sottosezione 3.1.3. Queste tre forme si riconoscono direttamente nel codice, e per questa parte la categoria si comporta come una categoria *tecnica*: il rilevatore è statico e lavora sui sorgenti Python. Resta fuori ciò che il codice non può dire, cioè se la fiducia mal riposta porti davvero a una compromissione: è un residuo di natura *dinamica* nel senso di RQ_{1,2}, ripreso nel Capitolo 4.

Le tre regole Il modulo percorre i file Python del bersaglio, li legge con l'analizzatore sintattico della libreria standard e applica tre regole, riassunte nella Tabella 3.8. La prima insegue il dato che arriva da un servizio esterno e viene poi accettato, salvato o passato ad altri componenti senza alcun controllo: è l'iniezione differita, dove un contenuto malevolo entra dal canale, ritenuto fidato, di una terza parte e fa danni più tardi, quando l'applicazione lo rilegge e lo usa (CWE-20 [51]). La seconda cerca le chiamate che raggiungono un servizio esterno su canale in chiaro¹⁷, cioè con `http` invece di `https` (CWE-319 [52]). La terza individua i client che seguono ciecamente le redirezioni¹⁸ indicate da una terza parte (CWE-601 [53]).

¹⁶*Confine di fiducia (trust boundary)*: la linea che separa la parte di un sistema di cui ci si fida da quella che va invece verificata; collocarlo nel punto sbagliato significa accettare senza controlli dati che provengono in realtà dall'esterno.

¹⁷*In chiaro (cleartext)*: senza cifratura; un URL `http://` anziché `https://` espone i dati in transito a chi intercetta il traffico, l'attacco *man-in-the-middle*.

¹⁸*Redirezione (redirect)*: la risposta con cui un server rimanda il client a un altro indirizzo; seguirla in automatico, con l'opzione `allow_redirects` attiva, consente a un fornitore compromesso di dirottare la richiesta, dati sensibili compresi, verso un server ostile.

Regola	Comportamento ricercato	Severità
UC-001	dati di terze parti usati senza validazione	HIGH
UC-002	consumo di un servizio esterno su canale in chiaro (<code>http</code>)	HIGH
UC-003	redirezioni di terze parti seguite ciecamente	HIGH

Tabella 3.8: Le tre regole del modulo, corrispondenti alle condizioni di consumo insicuro non già coperte dalla Sottosezione 3.4.1.

Le tre regole cercano segnali di natura diversa. La prima segue un dato. Di solito l'analisi del flusso dei dati parte dall'input dell'utente, mentre qui il dato da sorvegliare è la risposta di un servizio esterno: la regola prende quindi come punto di partenza il contenuto restituito da una chiamata HTTP in uscita, per esempio il risultato di `requests.get(...).json()`, e ne segue l'uso fino al punto in cui viene salvato o passato ad altri componenti senza controlli. La seconda guarda un valore: l'indirizzo del servizio chiamato, che non deve cominciare con `http://`. La terza guarda un'opzione che manca: `requests`, per esempio, segue le redirezioni in automatico a meno che la chiamata non contenga `allow_redirects=False`, e la regola segnala le chiamate verso terze parti in cui quell'opzione è assente.

Come nel modulo precedente, ogni segnalazione porta con sé un grado di confidenza, e quelle sotto la soglia di 0,70 non entrano nel rapporto. Il rapporto indica, per ciascuna regola, se ha prodotto almeno una segnalazione e riassume i risultati per severità e categoria.

CAPITOLO 4

Risultati

Questo capitolo riporta ciò che è emerso applicando i moduli del Capitolo 3 secondo il disegno della Sottosezione 3.1.5. Le evidenze sono di due tipi e vanno lette in modo diverso. Le coppie di casi di prova, scritte dallo stesso autore delle regole, dicono se ogni modulo riconosce la condizione per cui è stato costruito; il codice non scritto per i moduli, cioè i repository esterni e crAPI, mostra come si comporta lo strumento quando la condizione non è stata preparata per lui. Dopo aver descritto l'oggetto della valutazione, il capitolo presenta i risultati dei due tipi di prova, discute il comportamento delle categorie alla luce della classificazione e si chiude con i limiti emersi e con le minacce alla validità.

4.1 Oggetto della valutazione

I moduli non sono stati esercitati tutti nello stesso modo. La Tabella 4.1 riassume, per ciascuno, le prove di cui si riportano gli esiti; per i moduli che ne sono privi il capitolo discute soltanto i limiti emersi dall'uso.

Il bersaglio infrastrutturale è costituito da due manifest Terraform: cinque bucket S3 a protezione crescente, dall'ACL pubblica al bucket cifrato con blocco dell'accesso attivo, un sesto bucket con policy aperta, un ruolo IAM con * su *, una Lambda

Modulo	Casi di prova	Codice non scritto per il modulo
IaC misconfiguration	manifest Terraform con bucket a protezione crescente	crAPI (Dockerfile e manifest Kubernetes)
API1 – BOLA	coppia gemella in Flask ed Express, due controlli negativi	repository esterno etichettato; crAPI
API2 – Broken Authentication	–	–
API3 – BOPLA	coppia gemella sul profilo utente	crAPI
API4 – Unrestricted Resource Consumption	–	–
API5 – BFLA	coppia gemella con le sette debolezze	repository esterno con specifica OpenAPI
API7 – SSRF	–	–
API8 – Security Misconfiguration	–	–
API10 – Unsafe Consumption	coppia gemella	bersaglio realistico

Tabella 4.1: Prove di cui si riportano gli esiti, per modulo. Un trattino indica che per quel modulo non si riportano esiti sperimentali; i limiti che lo riguardano sono discussi nella Sezione 4.5.

con chiave cablata e un API Gateway senza autorizzazione. Il confronto previsto dal disegno è quello fra i primi tre bucket, esposti, e gli ultimi due, protetti.

Per la BOLA ogni regola statica ha un endpoint vulnerabile e la sua versione corretta, e ciascun caso è scritto sia in un servizio Flask sia in un servizio Express, ognuno con la propria specifica OpenAPI. Si aggiungono due controlli negativi: un profilo protetto da un confronto fra identificativo e identità, e una cancellazione riservata agli amministratori, che appartiene al dominio della BFLA. Il confronto con il riferimento avviene sulla combinazione di file, regola, metodo e percorso, così che una regola corretta sull'endpoint sbagliato conti come errore. La coppia della BOPLA modella invece un profilo utente in cui stipendio e ruolo spettano al solo amministratore: la versione vulnerabile restituisce il profilo per intero e copia sul modello ogni campo ricevuto, quella irrobustita mostra i due campi al solo amministratore e accetta soltanto i campi di un'allowlist, rispondendo comunque con successo agli altri, così da smascherare un modulo che si fidasse del codice di risposta. Nella coppia della BFLA l'applicazione vulnerabile contiene un esempio per

ciascuna delle sette debolezze e la versione irrobustita le corregge; in quest'ultima, per esempio, l'endpoint degli inviti verifica che il chiamante sia amministratore prima di assegnare un ruolo diverso da quello base, mentre un endpoint di ricerca usa il ruolo solo come filtro, così da mettere alla prova le esclusioni di BF-007. Per l'Unsafe Consumption, infine, si dispone di due applicazioni gemelle, una vulnerabile e una irrobustita.

Il livello statico della BOLA e la BFLA sono stati eseguiti anche su codice esterno: nel primo caso su un repository che dichiara nel codice quali endpoint siano vulnerabili e quali protetti, nel secondo su un repository fornito di specifica OpenAPI. crAPI [32] è stato fissato alla revisione 1.1.5, che non contiene codice Terraform ma include Dockerfile e manifest Kubernetes.

Accanto a queste prove, le prove d'unità verificano i singoli componenti. Per la BOLA sono 118, di cui 68 per il livello dinamico, con ZAP e le chiamate HTTP simulati, e 50 per quello statico; per la BFLA sono 61, di cui 33 dedicate a BF-007, con un caso vulnerabile per ogni tipo di assegnazione e un caso sicuro per ogni esclusione. Per la BOLA si è inoltre controllato che le prove verificano davvero le euristiche, introducendo una alla volta sette alterazioni mirate del codice, come la rimozione della pulizia dei commenti o della divisione delle condizioni composte.

4.2 Risultati sui casi di prova

Sul bersaglio Terraform Checkov riporta 86 controlli falliti su 24 risorse: i bucket sono infatti scomposti nelle rispettive sotto-risorse, cioè blocco dell'accesso pubblico, policy e controlli di proprietà. Il catalogo delle policy risolve tutti i controlli per mappatura esplicita, con confidenza 1, e ne classifica 33 come esposizioni e 53 come irrobustimenti. L'ordinamento dei punteggi di rischio rispetta quello atteso (Tabella 4.2): le risorse esposte precedono tutte le altre, e i due bucket protetti chiudono la graduatoria con i soli irrobustimenti. L'unico scostamento è silenzioso: il controllo CKV_AWS_70 non valuta la policy aperta del secondo bucket, perché l'attributo `Resource` è una lista di interpolazioni, e di conseguenza non compare né fra i controlli superati né fra i falliti.

Risorsa	Punteggio	Natura
ACL pubblica e policy aperta	9,2	esposizione
Chiave cablata (Lambda)	8,8	esposizione
Policy IAM * su *	8,6	esposizione
Metodi senza autorizzazione	7,4	esposizione
Blocchi di accesso disattivati	6,8	esposizione
Bucket sicuri (due)	3,2	solo irrobustimenti

Tabella 4.2: Punteggi di rischio delle voci del bersaglio Terraform, in ordine decrescente.

Sulla coppia gemella della BOLA il livello statico ha prodotto 12 veri positivi, nessun falso negativo e nessun falso positivo, senza segnalazioni sull'applicazione corretta. Quattro dei veri positivi riguardano operazioni che il contratto dichiara protette e sulle quali la regola di Spectral dedicata ad API1 non segnala nulla: è il punto cieco che il confronto con il contratto (Sottosezione 3.3.1) doveva coprire, e la prova mostra che lo copre. Tutte e sette le alterazioni del codice hanno fatto fallire almeno una prova d'unità; l'unica che inizialmente non ne faceva fallire nessuna ha portato all'aggiunta di un caso specifico.

In entrambe le versioni della coppia il modulo della BOPLA individua come protetti esattamente stipendio e ruolo. Sulla versione vulnerabile segnala la copia indiscriminata dei campi (S01), il profilo restituito per intero (S02) e il ruolo inviato dal client ma assente dal contratto (S04), e le prove dinamiche confermano tutte e tre le segnalazioni. Sulla versione irrobustita resta solo la S04, che dipende dal contratto, identico nelle due versioni; le prove dinamiche la smentiscono, perché il ruolo inviato dall'utente non viene salvato. Il caso mostra all'opera il rapporto fra i due livelli: una segnalazione che resta vera sul contratto viene ridimensionata dall'osservazione del sistema.

Il modulo della BFLA ha riconosciuto tutte e sette le debolezze dell'applicazione vulnerabile e non ha prodotto segnalazioni sulla versione irrobustita, con il cento per cento di veri positivi e nessun falso positivo. L'introduzione di BF-007 non ha modificato le sei segnalazioni già prodotte sull'applicazione vulnerabile. Sulle

applicazioni gemelle dell'Unsafe Consumption il risultato è altrettanto netto: le tre regole scattano tutte sulla versione vulnerabile e nessuna su quella irrobustita, con il cento per cento di veri positivi e nessun falso positivo.

Questi risultati dicono che l'implementazione fa ciò che si intendeva, e non molto di più: casi e regole hanno lo stesso autore, e un esito pieno su una coppia gemella non stabilisce ancora come lo strumento si comporti su codice scritto da altri. È ciò che le prove della sezione seguente cercano di osservare.

4.3 Risultati su codice non scritto per i moduli

Sul repository esterno il livello statico della BOLA ha prodotto una segnalazione OA-001 su ciascuno dei quattro metodi della risorsa vulnerabile e nessuna sulla risorsa protetta. Il risultato non era scontato: alla prima esecuzione anche la risorsa protetta veniva segnalata, perché lettura dell'identità e recupero dell'oggetto si trovavano in una funzione di supporto, ed è da questo caso che nasce il riassunto delle funzioni dello stesso file descritto nella Sottosezione 3.3.1. Uno dei metodi vulnerabili, inoltre, non compare nella specifica: il livello dinamico, che costruisce l'inventario dal contratto, non lo attaccherebbe, e solo l'analisi statica lo raggiunge. Un'ulteriore segnalazione, su un endpoint di importazione che permette a qualunque utente autenticato di sovrascrivere il proprietario di un progetto, non è coperta dalle etichette del repository e dovrà essere verificata con una scansione dinamica; fino ad allora non può essere contata né come vero né come falso positivo.

Sul repository fornito di specifica OpenAPI la BFLA ha prodotto tre segnalazioni BF-006, su endpoint che popolano, salvano e ripristinano lo stato dei dati senza richiedere né autenticazione né ruolo. Dal punto di vista tecnico sono veri positivi, ma quegli endpoint sono voluti: servono alle prove automatiche per preparare e ripulire i dati fra un'esecuzione e l'altra. Gli endpoint applicativi, che verificano proprietario o ruolo, non hanno prodotto segnalazioni. La stessa prova ha fatto emergere un difetto di presentazione: sul codice Python le regole BF-004 e BF-006 riportano sempre GET come metodo, anche quando la rotta ne usa un altro. Il percorso resta corretto e la deduplicazione, che raggruppa per percorso, non ne risente, ma il metodo mostrato può essere sbagliato.

Su crAPI il quadro cambia da modulo a modulo. Per la parte infrastrutturale le 240 segnalazioni prodotte su Dockerfile e manifest Kubernetes (17 esposizioni e 223 irrobustimenti) si riducono a 46 voci. In testa, con punteggio 7,0, si trovano i token di sessione presenti nella collezione Postman e le credenziali di una dipendenza inclusa nel repository; queste ultime costituiscono un falso positivo, poiché non appartengono a crAPI. Segue, con 6,8, la specifica OpenAPI priva di requisiti di sicurezza, mentre le voci relative a Kubernetes e Docker chiudono l’inventario con punteggi compresi fra 4,7 e 3,2. La prova ha inoltre fatto emergere un difetto dell’adapter, che assegnava la stessa severità a ogni segnalazione perché le regole operavano sull’identificativo opaco del controllo anziché sul suo nome ufficiale: il catalogo delle policy descritto nella Sottosezione 3.2.2 nasce proprio da questa correzione.

Il livello statico della BOLA, invece, su crAPI non offre alcuna copertura. I casi di BOLA di quell’applicazione si trovano in servizi scritti in Java, in Go e in Django con viste definite come classi, e in un caso l’identificativo arriva nella query anziché nel percorso: sono tutti fuori dal perimetro del livello statico, e su questo bersaglio la categoria resta affidata al solo livello dinamico. La BOPLA riconosce come protette le proprietà riservate ai meccanici e i campi che il contratto dichiara di sola lettura; la sua regola S01, però, non è in grado di vedere un mass assignment realizzato assegnando i campi uno per uno, come accade in crAPI.

Sul bersaglio realistico le regole dell’Unsafe Consumption non producono segnalazioni, un esito che riflette le caratteristiche architetture del codice esaminato più che un difetto di copertura. UC-002 e UC-003 identificano il servizio esterno a partire da un indirizzo statico presente nel sorgente, mentre lì gli endpoint delle terze parti sono definiti solo a runtime e restano fuori dal perimetro dell’analisi statica. UC-001, che non dipende dall’indirizzo ma segue il flusso del dato dalla risposta della terza parte fino al punto di riuso, non trova punti di iniezione non protetti, come query costruite per concatenazione o oggetti creati da dizionari non filtrati: manca il bersaglio dell’iniezione differita. Lo stesso codice resta coperto, da un’angolazione complementare, dal modulo per la Server-Side Request Forgery (Sottosezione 3.4.2). Proprio perché il difetto non c’è, però, questo bersaglio non dice nulla sulla sensibilità del modulo, che resta documentata dalle sole applicazioni gemelle.

4.4 Comportamento delle categorie

4.4.1 Categorie ibride

Le prove confermano la divisione del lavoro prevista dalla classificazione. La parte statica trova la condizione necessaria e raggiunge punti che la verifica dinamica non vedrebbe: i quattro veri positivi della BOLA su operazioni che il contratto dichiara protette, dove Spectral tace, e il metodo vulnerabile del repository esterno assente dalla specifica. La parte dinamica, dove c'è, restituisce ciò che il codice non può dire, come mostra la segnalazione S04 della BOPLA, vera sul contratto e smentita dal sistema in esecuzione. Il meccanismo con cui la BOLA porta conferme e smentite sulla stessa voce è però verificato soltanto dalle prove d'unità, e per il suo livello dinamico non si riportano esiti su un bersaglio in esecuzione: la segnalazione sull'endpoint di importazione del repository esterno, che attende proprio quella verifica, è l'esempio concreto di un residuo dinamico rimasto aperto.

Nella BFLA il residuo resta aperto per costruzione, e le prove mostrano quanto sarebbe difficile chiuderlo. OWASP giudica facile la rilevabilità della categoria, perché le API sono strutturate e il modo di raggiungere una funzione è prevedibile [19], ma il giudizio riguarda metà del problema: formulare la chiamata candidata è facile, stabilire se la risposta fosse dovuta non lo è, perché un 200 OK attesta che l'operazione è stata eseguita, non che fosse legittimo eseguirla. Il contributo della letteratura non consiste nell'ispezionare il codice, ma nel ricostruire la specifica mancante, ricavandola dalle pagine raggiungibili da ciascun ruolo [54], dalla coerenza con cui i controlli sono applicati dentro una partizione per ruolo [55] o dal contesto di autorizzazione [56]; un'alternativa rinuncia a inferire e scrive regole di sicurezza per le API REST, verificandole con fuzzing stateful [57], mentre un lavoro recente affronta lo stesso problema in modalità black box [58]. Le tre segnalazioni BF-006 sul repository esterno illustrano il punto: la regola stabilisce che quelle funzioni sono raggiungibili senza ruolo, non se debbano esserlo, e a deciderlo è chi conosce lo scopo degli endpoint.

Per la Broken Authentication non si riportano esiti misurati su casi di prova. L'esperienza d'uso del modulo ha messo in luce soprattutto i suoi limiti, discussi

nella Sezione 4.5, a partire dalla dipendenza della parte dinamica da una sessione autenticata.

4.4.2 Categorie a rilevazione statica

Per l'Unrestricted Resource Consumption, classificata come tecnica, non si riportano esiti sperimentali, ma il limite principale del modulo è di natura metodologica e merita una discussione a parte. Il modulo non vede l'API in funzione: quanto costi davvero un endpoint, o se un tetto dichiarato venga poi rispettato, si scopre solo mettendo il sistema sotto carico e osservandone i tempi di risposta e le chiamate verso l'esterno. Le due fonti primarie sembrano qui contraddirsi, perché OWASP giudica facile la rilevabilità, dato che richieste con parametri di dimensione crescente e l'osservazione di stato, tempo e lunghezza della risposta bastano a far emergere il problema [11], mentre CWE-770 giudica di utilità limitata l'analisi statica per le allocazioni prive di limite [15]. La contraddizione è apparente: OWASP descrive un tester in *black box*¹, CWE uno strumento che legge il codice. Sul versante dinamico l'evidenza è consolidata, dalla generazione di molte richieste in un intervallo breve al rilevamento per iniezione di carico [15, 59]; il modulo, lavorando per indizi, sa dire che un limite manca, ma non se la sua assenza basti a mettere in difficoltà il servizio. Una sua segnalazione va letta quindi come una protezione da aggiungere, e la severità dice quanto quella protezione conti, non quanto sia probabile un attacco.

L'Unsafe Consumption presenta una copertura mista. L'analisi statica ne individua la *predisposizione* — la validazione assente, il canale in chiaro, la redirectione seguita ciecamente — mentre la verifica della sfruttabilità effettiva richiede un piano dinamico che metta alla prova il sistema in esecuzione contro un fornitore ostile e controllato: un banco che osservi il traffico in uscita e risponda di proposito con dati malevoli o con una redirectione verso un host pericoloso. Una verifica del genere è possibile solo se il servizio reale consente di immettervi contenuti, oppure sostituendolo con un doppio in collaudo, che però verifica l'applicazione e non l'integrazione reale. Il residuo non coperto è quindi di natura dinamica, nel senso di RQ_{1,2}, e la

¹*Black box*: modalità di analisi che osserva il sistema solo dall'esterno, attraverso le risposte alle richieste, senza accesso al codice sorgente; vi si contrappone l'analisi *white box*, che legge il codice.

sua verifica andrebbe integrata con quella della Server-Side Request Forgery. Per la Server-Side Request Forgery e per la Security Misconfiguration non si riportano esiti sperimentali; i loro limiti sono discussi nella sezione seguente.

4.4.3 Categorie senza modulo

Due categorie restano prive di un modulo, e per ragioni diverse. Per API6:2023 il rilevamento automatico si scontra con l'assenza di un oracolo: nessun artefatto analizzabile, che sia codice, contratto o configurazione, risponde alla domanda se un dato flusso in una data azienda vada protetto, e la mancanza di una mappatura CWE registra il fatto che non esiste una debolezza del software da nominare. Le tecniche note per i difetti di logica applicativa cercano deviazioni da un comportamento atteso [30, 31], mentre qui il flusso viene percorso correttamente e l'attacco sta nel ripeterlo. Restano le contromisure comportamentali a runtime indicate da OWASP [23], che riconoscono l'automazione durante l'attacco anziché la vulnerabilità prima che avvenga. Nei termini del criterio, la categoria ricade quindi fra quelle di logica di business.

API9:2023 resta invece fuori dalla tripartizione, perché presuppone che il sistema da valutare non sia noto (Sottosezione 3.1.3). OWASP ne giudica media la rilevabilità senza indicare un metodo, limitandosi a osservare che una documentazione obsoleta rende più difficile individuare e correggere le vulnerabilità [22]: il difetto ostacola il proprio rilevamento, perché l'artefatto che servirebbe a trovarlo è quello che manca. Per via automatica si può soltanto confrontare rappresentazioni diverse dello stesso sistema: le rotte implementate nel codice ma non dichiarate nel contratto sono candidate a essere API ombra, quelle dichiarate ma non implementate segnalano documentazione obsoleta, e nessuna delle due è una vulnerabilità. Segnali di questo tipo compaiono nel sistema solo come effetto collaterale di altri moduli, nell'handler della BOLA assente dal contratto e nella funzione ombra della BFLA, e non costituiscono una copertura della categoria. Il confronto ha del resto un limite intrinseco, perché opera fra due rappresentazioni della stessa applicazione e non dice nulla sugli host su cui essa è distribuita: il primo scenario OWASP è precisamente il caso in cui codice e contratto coincidono e il difetto sta nella topologia di rete, il che richiede

ricognizione infrastrutturale e non analisi del software.

4.5 Limiti emersi dalla valutazione

I limiti emersi si raggruppano in pochi temi, che attraversano i moduli più di quanto li distinguano. Nessuno tocca il criterio di classificazione; tutti delimitano ciò che lo strumento, così com'è, può affermare.

Presenza e correttezza di un controllo Il limite più generale è che l'analisi statica accerta la presenza di un controllo e non la sua correttezza, ed è esattamente il residuo che la classificazione assegna alle categorie ibride. Nella BOLA un filtro sul campo sbagliato viene accettato come legame, e una funzione chiamata `authorize` viene accettata senza esaminarne il contenuto. Nella BFLA il controllo di ruolo scritto nel corpo della funzione è riconosciuto per corrispondenza testuale, così che anche una riga che cita il ruolo senza bloccare l'esecuzione, come un messaggio di log, viene scambiata per una protezione; nel contratto, inoltre, qualsiasi schema di sicurezza basta a sopprimere BF-001, anche quando dichiara solo un'autenticazione e non un'autorizzazione. Il modulo verifica che il controllo di funzione ci sia, non che funzioni, e per accertarlo servirebbe invocare le funzioni privilegiate con le credenziali di un utente ordinario e osservarne l'esito.

Riconoscimento per nomi e forme d'uso Identità, controlli e funzioni riservate sono riconosciuti attraverso nomi e forme d'uso diffusi nei framework più comuni, e ciò che se ne discosta può sfuggire. Nella BOLA identità, funzioni di verifica e campi che indicano il proprietario sono riconosciuti per nome. Nella BFLA il riconoscimento di un endpoint privilegiato si basa su parole chiave nei percorsi e nei nomi delle funzioni, per cui una funzione riservata con un nome anonimo può sfuggire e un percorso dal nome allarmante ma legittimo può essere segnalato a torto. Nella Broken Authentication il riconoscimento dell'MFA e della limitazione del traffico si fonda su parole chiave e librerie note, un'euristica dichiarata esposta a mancati riconoscimenti su implementazioni fuori dai pattern attesi. La BOPLA riconosce i controlli di ruolo nelle forme più diffuse, ma non in tutte. Il modulo

dell'Unrestricted Resource Consumption, infine, non conosce il valore giusto di un limite per una data applicazione e se ne accorge soltanto quando è palesemente fuori scala; il suo confronto fra codice e contratto riconosce le forme più comuni con cui si dichiarano le rotte, e l'unica difesa di configurazione che oggi compensa una segnalazione è il tetto alla dimensione delle richieste. Se un endpoint ha un nome neutro, riceve dati generici e risponde in modo ordinario, dal contratto non si capisce che dietro c'è un servizio a pagamento, e lo rivela solo il codice, quando è disponibile.

Perimetro e granularità dell'analisi Diversi falsi positivi e mancati riconoscimenti dipendono dall'unità di codice esaminata. Il livello statico della BOLA analizza un file alla volta, e un controllo di proprietà contenuto in una funzione importata da un altro modulo, in un middleware definito altrove o in un livello di servizio non viene visto, producendo un falso positivo. BF-007 non segue il dato oltre i confini della funzione, per esempio dentro un servizio o un serializzatore, e può segnalare a torto un endpoint di registrazione che limita i ruoli ammessi a un elenco fisso senza interrogare il chiamante; allo stesso modo, nell'Unrestricted Resource Consumption i dati inviati dall'utente vengono seguiti solo all'interno di una singola funzione. Quanto ai linguaggi, le viste definite come classi, tipiche di Django REST Framework e di NestJS, non sono ancora analizzate dalla BFLA, che non tratta neppure l'autorizzazione fra gruppi o organizzazioni diverse; la BOPLA non analizza il codice Java e la sua regola S01 vede il mass assignment solo quando il corpo della richiesta viene copiato in blocco. Il caso di crAPI mostra quanto questi confini pesino su un bersaglio realistico. Per la Server-Side Request Forgery, l'analisi non verifica la raggiungibilità della destinazione a runtime, non rileva esfiltrazioni fuori banda (*Blind SSRF*) e non copre flussi che attraversano code o broker asincroni. Una verifica a runtime dovrebbe del resto distinguere le due forme dell'attacco: quella con ritorno si conferma puntando l'endpoint verso un indirizzo controllato, mentre quella cieca richiede un canale laterale, tipicamente una risoluzione DNS.

Analisi statica della configurazione Le prove sulla parte infrastrutturale e la natura statica del modulo per la Security Misconfiguration mettono in luce limiti comuni a ogni verifica di configurazione sugli artefatti. Un controllo che non riesce a valutare

un'espressione tace anziché fallire, e il silenzio di `CKV_AWS_70` sarebbe passato inosservato senza un riferimento di verità costruito a mano. Il catalogo delle policy va aggiornato a ogni nuova versione di Checkov, altrimenti i controlli aggiunti ricadono sulle regole generiche o sul ripiego predefinito. Il manifest mostra ciò che dichiara, non ciò che è effettivamente attivo, e una risorsa modificata a mano dalla console del provider resta invisibile; più in generale, ciò che non è dichiarato in un file non è ispezionabile prima dell'esecuzione. Non analizzando traffico reale, il modulo per la Security Misconfiguration non può individuare problemi legati ai server web o ai proxy intermedi, come la desincronizzazione delle richieste HTTP o l'abilitazione di metodi pericolosi a livello server come il verbo `TRACE`, e controlla le librerie su un archivio locale offline, lasciando l'aggiornamento continuo a strumenti dedicati. Il limite più sottile riguarda il contesto: il flag di esposizione dice che una risorsa è raggiungibile da chiunque, non se debba esserlo, e un bucket che serve i contenuti statici di un sito pubblico viene segnalato esattamente come uno che espone dati riservati; allo stesso modo una politica CORS che ammette qualunque origine è un difetto per un'API a client autenticati e una scelta legittima per un'API pubblica di sola lettura. Le segnalazioni BF-006 sul repository esterno ripropongono lo stesso limite sul piano delle funzioni.

La parte dinamica La parte dinamica, dove esiste, richiede di poter agire sul bersaglio con identità autenticate. Nella Broken Authentication la profondità della verifica dipende dalla possibilità di ottenere una sessione: dove le credenziali non sono disponibili o l'accesso è delegato a terzi, la parte dinamica si ritrae e resta soltanto la componente statica. Nella BOPLA le credenziali mancano nel cruscotto e nella scansione complessiva, per cui le segnalazioni restano da verificare. Il livello dinamico della BOLA richiede in Lab Mode un bersaglio cooperante, e una scansione con salvataggio e ripristino dello stato dura decine di minuti; il confronto delle risposte presume corpi in formato JSON, le parole che indicano un rifiuto sono un elenco fisso in italiano e in inglese, e il controllo anonimo segnala come difetto anche gli endpoint pubblici per scelta progettuale. L'uso di un modello linguistico nella prima fase della Broken Authentication introduce infine una fonte di non determinismo, contenuta dalla precedenza data alle euristiche e dalla degradazione controllata

quando il modello è assente.

Integrazione nel sistema L'integrazione dei moduli con il resto del sistema è ancora parziale. Il meccanismo di correlazione della BOLA è verificato dalle prove d'unità, ma il rilevatore statico non è ancora registrato nei runner della pipeline, per cui le sue segnalazioni non arrivano al cruscotto, e il catalogo per endpoint della riga di comando mostra una segnalazione smentita come potenziale anziché come sicura. La Broken Authentication opera come pipeline autonoma, con un proprio rapporto che contiene il grafo di conoscenza e gli esiti delle prove: una scelta che ne isola la logica, ma che tiene le sue segnalazioni distinte dal motore di correlazione, come accade per gli altri moduli presentati nella panoramica del cruscotto (Sezione 3.3).

Il prezzo del criterio L'ultimo limite discende direttamente dal criterio che vieta di indicare all'analisi ciò che va protetto. Nella BOPLA una proprietà è protetta solo se l'applicazione la tratta come tale in qualche punto, e un dato sensibile restituito a tutti senza alcun controllo di ruolo non viene segnalato. È una conseguenza voluta: un elenco fisso di campi sensibili avrebbe funzionato sulle applicazioni di prova e su nessun altro sistema.

4.6 Minacce alla validità

La minaccia principale alla validità interna è quella già considerata nel disegno della valutazione (Sottosezione 3.1.5): casi di prova e regole hanno lo stesso autore, e i risultati pieni sulle coppie gemelle attestano la conformità dell'implementazione alle intenzioni, non la sua efficacia su codice reale. Le esecuzioni su codice esterno la attenuano solo in parte, perché riguardano un repository per la BOLA e uno per la BFLA, e in entrambi i casi il riferimento di verità si è rivelato incompleto o ambiguo: la segnalazione sull'endpoint di importazione non è coperta dalle etichette del repository, e le funzioni di popolamento segnalate dalla BFLA sono veri positivi tecnici ma esposizioni volute. Il conteggio dei veri e dei falsi positivi dipende quindi da che cosa si decide di considerare vulnerabile.

Una seconda minaccia riguarda i componenti dinamici. Le 68 prove d'unità del livello dinamico della BOLA simulano ZAP e le chiamate HTTP, e il meccanismo con cui conferme e smentite raggiungono la stessa voce è verificato solo a quel livello; per la BOLA, a differenza della BOPLA, non si riportano esiti di prove dinamiche su un bersaglio in esecuzione. Sulla riproducibilità pesa inoltre il modello linguistico usato dalla Broken Authentication, mentre la revisione fissa di crAPI e la versione dichiarata di Checkov la sostengono.

Quanto alla validità esterna, crAPI resta un bersaglio didattico, costruito perché le sue vulnerabilità siano riconoscibili, e le sue caratteristiche — servizi in Java e in Go, viste Django definite come classi, endpoint delle terze parti definiti a runtime — lo collocano in larga parte fuori dal perimetro dei livelli statici. I risultati ottenuti su di esso, come quelli sui repository esterni, non si possono quindi estendere senza cautela ad applicazioni reali, né l'assenza di segnalazioni su un bersaglio che non contiene il difetto può essere letta come prova della sensibilità di un modulo.

CAPITOLO 5

Conclusioni

Le conclusioni qui...

Bibliografia

- [1] OWASP Foundation, “A05:2021 – security misconfiguration,” https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/, 2021, oWASP Top 10:2021. [Online]. Available: https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/ (Citato alle pagine 4 e 5)
- [2] SentinelOne, “Cloud misconfigurations: Risks, examples & prevention,” <https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-misconfigurations/>, 2025. [Online]. Available: <https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-misconfigurations/> (Citato a pagina 5)
- [3] Cloud Computing News, “10 real-life cloud security failures and what we can learn from them,” <https://www.cloudcomputing-news.net/news/10-real-life-cloud-security-failures-and-what-we-can-learn-from-them/>, 2025. [Online]. Available: <https://www.cloudcomputing-news.net/news/10-real-life-cloud-security-failures-and-what-we-can-learn-from-them/> (Citato a pagina 5)
- [4] OWASP Foundation, “OWASP API security top 10 – 2023,” <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>, 2023. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/> (Citato alle pagine 6, 16 e 35)

- [5] OWASP API Security Project, “API1:2023 broken object level authorization,” <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/> (Citato alle pagine 8, 21 e 49)
- [6] —, “API2:2023 Broken Authentication,” <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/> (Citato alle pagine 15, 16, 20 e 54)
- [7] OWASP Foundation, “Credential Stuffing,” https://owasp.org/www-community/attacks/Credential_stuffing, oWASP Community Pages. [Online]. Available: https://owasp.org/www-community/attacks/Credential_stuffing (Citato alle pagine 15 e 54)
- [8] Y. Sheffer, D. Hardt, and M. Jones, “JSON Web Token Best Current Practices,” RFC 8725, Internet Engineering Task Force (IETF), BCP 225, feb 2020. (Citato a pagina 16)
- [9] D. Temoshok, D. Proud-Madruga, Y.-Y. Choong, R. Galluzzo, S. Gupta, C. LaSalle, N. Lefkovitz, and A. Regenscheid, “Digital identity guidelines: Authentication and authenticator management,” National Institute of Standards and Technology, NIST Special Publication 800-63B-4, jul 2025. (Citato alle pagine 16 e 20)
- [10] OWASP Foundation, “Authentication Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html, oWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Citato a pagina 16)
- [11] OWASP API Security Project, “API4:2023 Unrestricted Resource Consumption,” <https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/>, 2023, consultato il 25 agosto

2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/> (Citato alle pagine 19, 20, 64 e 80)
- [12] A. Cha, E. Wittern, G. Baudart, J. C. Davis, L. Mandel, and J. A. Laredo, "A principled approach to GraphQL query cost analysis," in *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*. ACM, 2020, pp. 257–268. (Citato alle pagine 19, 20 e 31)
- [13] S. A. Crosby and D. S. Wallach, "Denial of service via algorithmic complexity attacks," in *Proceedings of the 12th USENIX Security Symposium*. Washington, DC: USENIX Association, 2003, pp. 29–44. (Citato a pagina 19)
- [14] C.-A. Staicu and M. Pradel, "Freezing the web: A study of ReDoS vulnerabilities in JavaScript-based web servers," in *Proceedings of the 27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, 2018, pp. 361–376. (Citato alle pagine 19 e 20)
- [15] MITRE Corporation, "CWE-770: Allocation of resources without limits or throttling," <https://cwe.mitre.org/data/definitions/770.html>, 2026, cWE versione 4.20. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/770.html> (Citato alle pagine 20 e 80)
- [16] M. Nottingham and R. T. Fielding, "Additional HTTP status codes," Internet Engineering Task Force, RFC 6585, apr 2012. (Citato a pagina 20)
- [17] R. Polli, A. Martinez Ruiz, and D. Miller, "RateLimit header fields for HTTP," IETF HTTPAPI Working Group, Internet-Draft draft-ietf-httpapi-ratelimit-headers-11, may 2026, work in progress. <https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/>. (Citato a pagina 20)
- [18] R. Chandramouli, "Security strategies for microservices-based application systems," National Institute of Standards and Technology, NIST Special Publication 800-204, aug 2019. (Citato a pagina 20)
- [19] OWASP API Security Project, "API5:2023 broken function level authorization," <https://owasp.org/API-Security/editions/2023/en/>

- 0xa5-broken-function-level-authorization/, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/> (Citato alle pagine 21, 22, 60 e 79)
- [20] MITRE Corporation, “CWE-285: Improper Authorization,” <https://cwe.mitre.org/data/definitions/285.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/285.html> (Citato alle pagine 21, 31 e 60)
- [21] OWASP API Security Project, “API8:2023 security misconfiguration,” <https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/> (Citato alle pagine 22, 26, 27, 68 e 69)
- [22] —, “API9:2023 Improper Inventory Management,” <https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/> (Citato alle pagine 22, 28, 29 e 81)
- [23] —, “API6:2023 Unrestricted Access to Sensitive Business Flows,” <https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/> (Citato alle pagine 23, 24, 31, 34 e 81)
- [24] —, “API7:2023 server side request forgery,” <https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/> (Citato alle pagine 25, 66 e 67)

- [25] MITRE Corporation, “CWE-918: Server-side request forgery (SSRF),” <https://cwe.mitre.org/data/definitions/918.html>, 2026, cWE versione 4.20. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/918.html> (Citato alle pagine 25 e 67)
- [26] —, “CWE-16: Configuration,” <https://cwe.mitre.org/data/definitions/16.html>, 2026, categoria CWE, versione 4.20; mappatura vietata. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/16.html> (Citato a pagina 26)
- [27] —, “CWE-1059: Insufficient technical documentation,” <https://cwe.mitre.org/data/definitions/1059.html>, 2026, cWE versione 4.20; denominazione precedente *Incomplete Documentation* fino ad aprile 2022; mappatura vietata. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/1059.html> (Citato alle pagine 28 e 31)
- [28] OWASP API Security Project, “Release notes – OWASP API security top 10 2023,” <https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/> (Citato a pagina 29)
- [29] —, “API10:2023 Unsafe Consumption of APIs,” <https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/> (Citato alle pagine 29, 30, 31 e 70)
- [30] V. Felmetsger, L. Cavedon, C. Kruegel, and G. Vigna, “Toward automated detection of logic vulnerabilities in web applications,” in *Proceedings of the 19th USENIX Security Symposium (USENIX Security 10)*. Washington, DC: USENIX Association, 2010, pp. 143–160. (Citato alle pagine 33 e 81)
- [31] G. Pellegrino and D. Balzarotti, “Toward black-box detection of logic flaws in web applications,” in *Proceedings of the 21st Network and Distributed System*

- Security Symposium (NDSS 2014)*, San Diego, CA, 2014. (Citato alle pagine 33 e 81)
- [32] OWASP Foundation, “crAPI: completely ridiculous API,” <https://github.com/OWASP/crAPI>, 2026. [Online]. Available: <https://github.com/OWASP/crAPI> (Citato alle pagine 37 e 75)
- [33] Prisma Cloud, “Checkov – what is checkov?” <https://www.checkov.io/>, 2026, analizzatore statico per Infrastructure-as-Code. [Online]. Available: <https://www.checkov.io/> (Citato a pagina 41)
- [34] HashiCorp, “Terraform documentation,” <https://developer.hashicorp.com/terraform/docs>, 2026. [Online]. Available: <https://developer.hashicorp.com/terraform/docs> (Citato a pagina 41)
- [35] M. Brunsfeld *et al.*, “Tree-sitter: an incremental parsing system for programming tools,” <https://tree-sitter.github.io/tree-sitter/>, 2026. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/> (Citato a pagina 48)
- [36] Stoplight, Inc., “Spectral – api style guide linter,” <https://docs.stoplight.io/docs/spectral>, 2026, linter per specifiche OpenAPI e AsyncAPI. [Online]. Available: <https://docs.stoplight.io/docs/spectral> (Citato a pagina 51)
- [37] Keycloak, “Keycloak documentation,” <https://www.keycloak.org/documentation>, 2026. [Online]. Available: <https://www.keycloak.org/documentation> (Citato a pagina 52)
- [38] ZAP Dev Team, “ZAP – zed attack proxy documentation,” <https://www.zaproxy.org/docs/>, 2026. [Online]. Available: <https://www.zaproxy.org/docs/> (Citato a pagina 52)
- [39] J. Yang, E. Wang, J. Chen, Q. Wang, Y. Zhang, H. Duan, W. Xie, and B. Wang, “Token Time Bomb: Evaluating JWT Implementations for Vulnerability Discovery,” in *Proceedings of the 32nd Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2025. (Citato a pagina 54)

- [40] B. Xu, S. Jia, J. Lin, F. Zheng, Y. Ma, L. Liu, X. Gu, and L. Song, "JWTKey: Automatic Cryptographic Vulnerability Detection in JWT Applications," in *Computer Security – ESORICS 2023*, ser. Lecture Notes in Computer Science, vol. 14346. Cham: Springer, 2023, pp. 263–282. (Citato a pagina 54)
- [41] OWASP API Security Project, "API3:2023 Broken Object Property Level Authorization," <https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/> (Citato a pagina 57)
- [42] MITRE Corporation, "CWE-862: Missing Authorization," <https://cwe.mitre.org/data/definitions/862.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/862.html> (Citato a pagina 60)
- [43] —, "CWE-650: Trusting HTTP Permission Methods on the Server Side," <https://cwe.mitre.org/data/definitions/650.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/650.html> (Citato a pagina 60)
- [44] —, "CWE-284: Improper Access Control," <https://cwe.mitre.org/data/definitions/284.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/284.html> (Citato a pagina 61)
- [45] —, "CWE-276: Incorrect Default Permissions," <https://cwe.mitre.org/data/definitions/276.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/276.html> (Citato a pagina 61)
- [46] —, "CWE-269: Improper Privilege Management," <https://cwe.mitre.org/data/definitions/269.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/269.html> (Citato a pagina 62)
- [47] M. Wessels, S. Koch, G. Pellegrino, and M. Johns, "SSRF vs. developers: A study of SSRF-defenses in PHP applications," in *Proceedings of the 33rd USENIX Security*

- Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, 2024, pp. 6777–6794. (Citato a pagina 67)
- [48] Y. Ji, T. Dai, Z. Zhou, Y. Tang, and J. He, “Artemis: Toward accurate detection of server-side request forgeries through LLM-assisted inter-procedural path-sensitive taint analysis,” *Proceedings of the ACM on Programming Languages*, 2025, preprint disponibile come arXiv:2502.21026. (Citato a pagina 67)
- [49] E. Wang *et al.*, “Where URLs become weapons: Automated discovery of SSRF vulnerabilities in web applications,” in *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P 2024)*. IEEE, 2024. (Citato a pagina 67)
- [50] OWASP API Security Project, “API10:2023 unsafe consumption of APIs – conflicting vulnerability indicators,” <https://github.com/OWASP/API-Security/issues/80>, 2023, issue n. 80 del repository GitHub OWASP/API-Security, marzo 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://github.com/OWASP/API-Security/issues/80> (Citato a pagina 71)
- [51] MITRE Corporation, “CWE-20: Improper Input Validation,” <https://cwe.mitre.org/data/definitions/20.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/20.html> (Citato a pagina 71)
- [52] —, “CWE-319: Cleartext Transmission of Sensitive Information,” <https://cwe.mitre.org/data/definitions/319.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/319.html> (Citato a pagina 71)
- [53] —, “CWE-601: URL Redirection to Untrusted Site (Open Redirect),” <https://cwe.mitre.org/data/definitions/601.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/601.html> (Citato a pagina 71)
- [54] F. Sun, L. Xu, and Z. Su, “Static detection of access control vulnerabilities in web applications,” in *Proceedings of the 20th USENIX Security Symposium (USENIX Security 11)*. San Francisco, CA: USENIX Association, 2011. (Citato a pagina 79)

- [55] S. Son, K. S. McKinley, and V. Shmatikov, “RoleCast: Finding missing security checks when you do not know what checks are,” in *Proceedings of the 2011 ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2011)*. ACM, 2011, pp. 1069–1084. (Citato a pagina 79)
- [56] M. Monshizadeh, P. Naldurg, and V. N. Venkatakrishnan, “MACE: Detecting privilege escalation vulnerabilities in web applications,” in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS 2014)*. ACM, 2014, pp. 690–701. (Citato a pagina 79)
- [57] V. Atlidakis, P. Godefroid, and M. Polishchuk, “Checking security properties of cloud service REST APIs,” in *Proceedings of the 13th IEEE International Conference on Software Testing, Validation and Verification (ICST 2020)*. IEEE, 2020, pp. 387–397. (Citato a pagina 79)
- [58] F. Liu, Y. Zhang, E. Li, W. Meng, Y. Shi, Q. Wang, C. Wang, Z. Lin, and M. Yang, “BACScan: Automatic black-box detection of broken-access-control vulnerabilities in web applications,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS 2025)*. ACM, 2025, pp. 1320–1333. (Citato a pagina 79)
- [59] J. Antunes, N. F. Neves, and P. Veríssimo, “Detection and prediction of resource-exhaustion vulnerabilities,” in *Proceedings of the 19th IEEE International Symposium on Software Reliability Engineering (ISSRE 2008)*. IEEE, 2008. (Citato a pagina 80)

Ringraziamenti

Ringraziamenti qui...